

GatherGo-揪人不糾結



四技 第115410組



指導教授：蒯思齊 老師



組長：11246080 蕭安庭

組員：11246061 鄭喬尹

11246066 賴若禾

11246069 吳文鈺

11246079 李若妘



2026/6/12





你是不是也常常和朋友約出門，卻總是不知道要去哪裡？

研究動機與目標

研究動機

朋友相約聚餐或規劃出遊時，常在 **LINE** 群組討論，資訊分散、條件繁多，導致討論冗長、決策效率低。

我們希望讓系統主動理解對話情境，提供即時且符合需求的建議，提升決策效率與便利性。

01

資訊來源分散

餐廳、景點、交通與預算資訊分散在不同平台，需要花時間搜尋與整合。

02

討論冗長難以決定

多人意見難以整合，條件不斷變動，反覆討論仍難達成共識。

03

決策效率低

耗費大量時間在找資料與比較，影響聚餐或出遊的規劃效率。

04

缺乏即時決策輔助

現有工具無法從對話中理解需求，僅能被動查詢，缺乏主動建議。

研究目的

本研究旨在開發一套基於情境感知之 **LINE Bot** 智慧推薦系統，提升群組聚餐與出遊討論的決策效率與便利性。

01

自動分析群組對話

從群組聊天內容中辨識需求與條件，理解聚餐或出遊的討論情境。

02

智慧推薦餐廳與景點

依據使用者條件與偏好，推薦最合適的餐廳與景點選擇。

03

整合行程規劃與預算分析

提供路線安排與預算估算，協助使用者快速完成行程決策。

< 好友群組 (6) 🔍 📞 ☰

A 這週要不要吃飯 12:30

B 可以啊 12:31

C 不要太遠 12:31

D 捷運站附近比較好 12:31

E 大概4個人 12:32

F 想找可以聊天久坐的 12:32

LINE Bot :
已分析需求, 正在整理最佳方案... 12:33

偵測到的需求條件

-  預算：未明確（可補充）
-  地點：捷運站附近（步行5分鐘內）
-  人數：約4人
-  偏好：適合聊天久坐
-  活動：聚餐（吃飯）

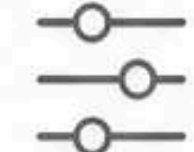
🔄 條件持續動態更新

★ 重點

 系統透過語意分析自動理解群組對話，辨識**活動類型**（吃飯／逛街／看電影）、**人數**、**地點條件**與**使用者偏好**，並在適當時機提供**決策建議**與**行程規劃**。

🧠 AI 行為

 **意圖辨識**
(吃飯／出遊／電影)

 **條件收斂**
(時間／地點／人數)

 **情境判斷**
(是否介入)

 **自動推薦**
(當資訊足夠時)

意圖辨識 (劇本二) ▶ 條件收斂 (劇本三) ▶ 行程生成 (劇本四)

< 好友群組 (6) 🔍 📞 ☰

A 這週要不要吃飯? 12:30

B 可以啊 12:31

C 餐廳不要太遠 12:31

D 捷運站附近五分鐘內 12:31

E 想吃韓式或日式 12:32

F 最近有點窮，不要太貴，300元內 🥺 12:33

LINE Bot :
已理解需求，以下推薦符合條件的餐廳與景點給大家參考！ 12:33

推薦餐廳 (符合條件)

 1. 韓式料理 (捷運站附近)

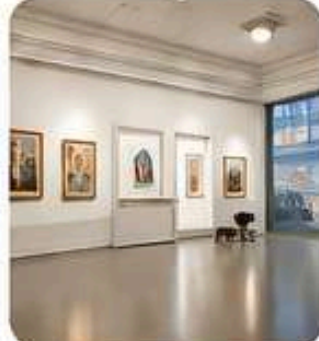
- 步行約3分鐘
- 評價：4.6 ★
- 價格：約250-300元
- 環境：安靜、適合聊天久坐

 2. 日式定食 (捷運站附近)

- 步行約4分鐘
- 評價：4.5 ★
- 價格：約280元
- 環境：安靜、氣氛佳

💡 Bot 提示：兩間餐廳皆在捷運站步行5分鐘內，價格符合預算，環境適合聊天久坐！

推薦景點 (符合條件)

 室內展覽館

- 步行 + 捷運約15分鐘
- 門票：\$100
- 適合拍照打卡
- 冷氣舒適、雨天備案佳

 2. 文青咖啡廳

- 步行約8分鐘
- 門票：免費 (低消制)
- 適合拍照打卡、氣氛佳
- 安靜、適合聊天久坐

💡 Bot 提示：兩個景點距離近，適合安排在餐後行程，拍照打卡效果佳！

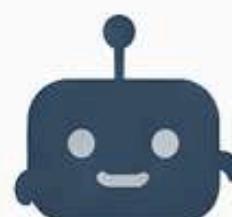


★ 重點

系統依據使用者條件與偏好，自動搜尋並過濾符合的餐廳與景點，同時提供環境、價格、距離等資訊，幫助使用者快速找到最適合的選擇，為後續路線規劃與預算估算奠定基礎。

🧠 AI 行為

- 條件匹配 (地點 / 預算 / 偏好 / 類型)
- 資訊整合 (評價 / 距離 / 價格)
- 多方案推薦 (餐廳與景點)

預算分析（每人）		
	餐費	約250～300元
	已選擇符合預算的餐廳	
	交通費	約40～60元
	- 捷運來回：約40元 - 步行為主，無額外交通費	
	景點門票	約100元
	室內展覽館門票	
總計預估		約390～460元 / 人
	在預算（300元內）附近，符合經濟需求	
 Bot 提示： 可依需求調整咖啡廳或景點，進一步降低預算！		





問卷調查與研究目的

本研究針對群組聚餐與出遊情境進行問卷調查，
以了解使用者在討論過程中的**常見問題、**
需求與決策痛點，
作為系統功能設計與研究驗證依據。



研究目的

透過問卷蒐集使用者真實意見，
掌握群組討論中的需求與痛點，
打造更符合使用情境的智慧輔助系統。

問卷調查概況



發放數

150
份



有效回收數

113
份



回收率

75.3%

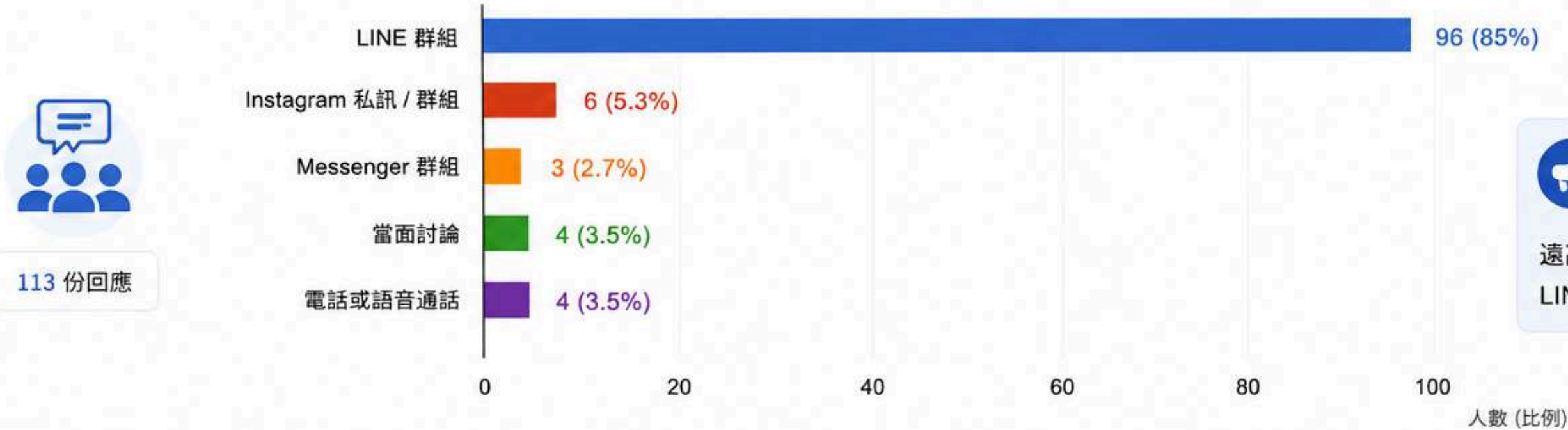


有效回收 113 份 (75.3%)

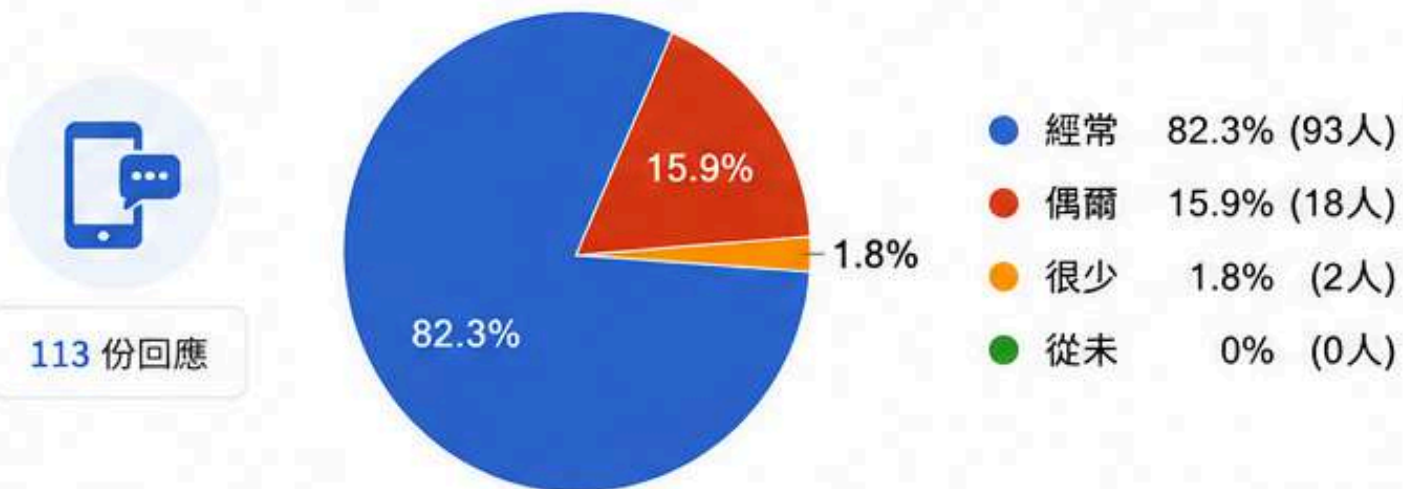
未回收 37 份 (24.7%)

註：本問卷調查對象為曾參與群組聚餐或出遊討論之使用者。

5. 當您與朋友討論聚餐或出遊時，最常使用哪個方式進行討論？

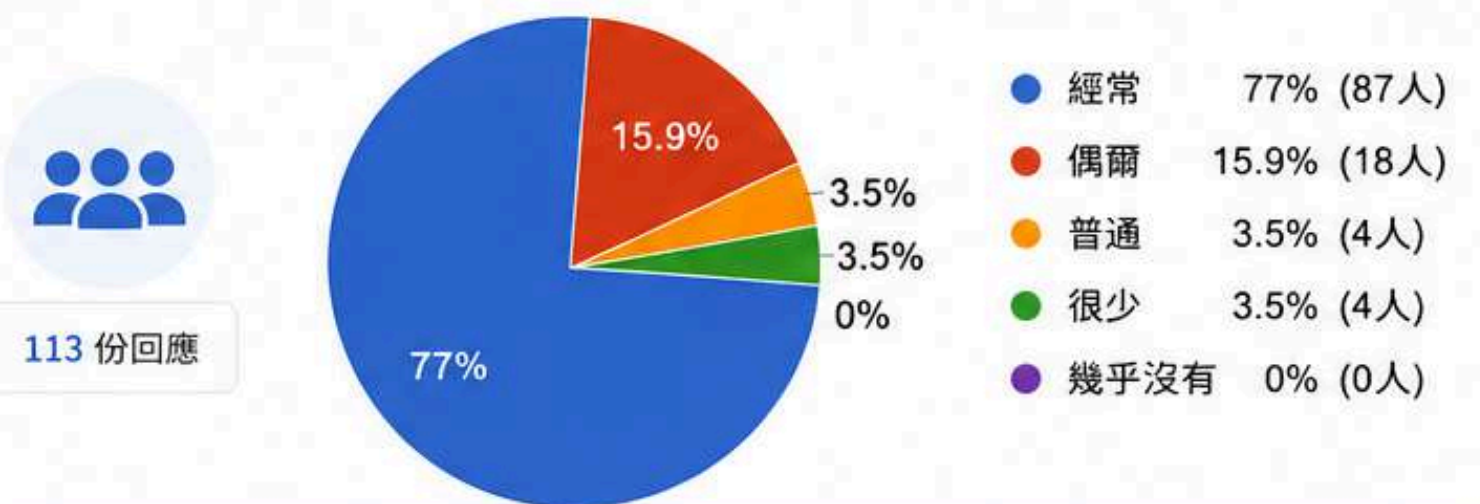


6. 您是否曾使用通訊軟體（如 LINE）與他人討論旅遊或聚餐規劃？



✓ 高達 **98.2%** 的受訪者曾使用 LINE 與他人討論旅遊或聚餐規劃，顯示通訊軟體已是日常討論的重要工具。

7. 您平常是否會透過群組與朋友討論聚餐或出遊安排？



✓ 超過 **92.9%** 的受訪者會透過群組討論聚餐或出遊安排，群組討論已成為決策流程中的常態行為。



綜合發現

本研究顯示，大多數使用者習慣使用 LINE 群組進行聚餐與出遊的討論與決策。通訊平台已成為主要的決策場域，顯示具備整合推薦、資訊整理與即時協助功能的群組系統具有高度實際需求與發展價值。



8. 討論時曾遇到哪些情況？(可複選)

113 份回應



超過五成受訪者表示「預算不同」與「選項太多難以決定」是最常遇到的問題，討論過程耗時且需要反覆比較資訊。



9. 最容易影響聚餐或出遊決定的因素

113 份回應

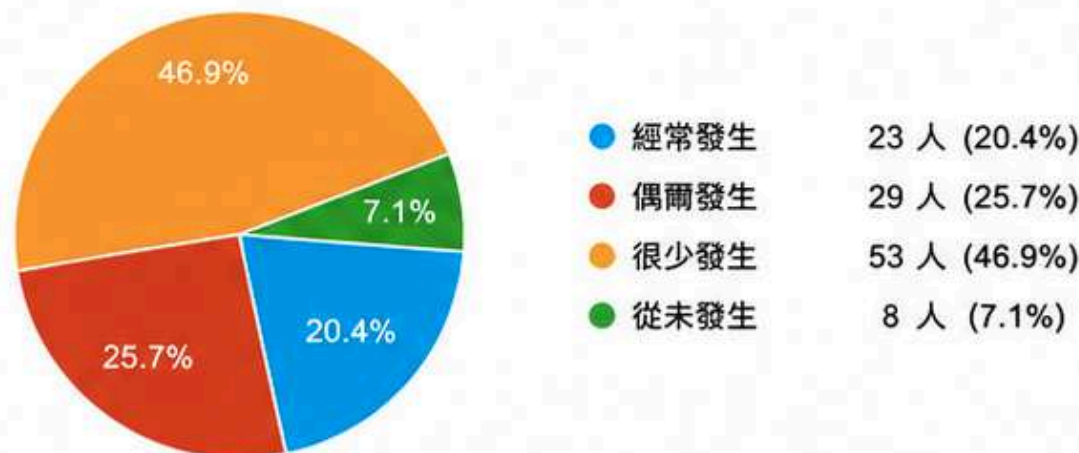


「距離與交通便利性」與「預算差異」為主要影響因素，顯示行程安排需兼顧便利性與預算考量。



10. 是否曾因預算不同而影響行程安排或最終決定？

113 份回應

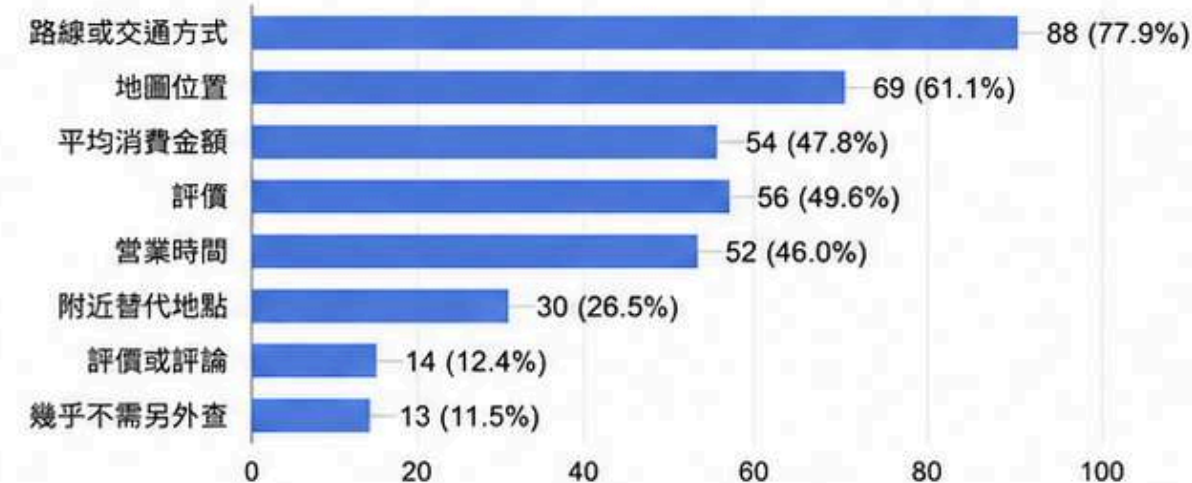


超過 7 成受訪者表示曾因預算不同而影響行程，顯示預算差異是群組決策的重要痛點。



11. 地點確定後，通常還會查詢哪些資訊？(可複選)

113 份回應



多數使用者在地點確定後仍需查詢「交通方式」、「地圖位置」、「消費金額」與「評價」等資訊，資訊分散且查找耗時。



綜合分析

問卷結果顯示，使用者在聚餐與出遊決策過程中，常面臨資訊分散、意見難以整合、預算差異與路線規劃等問題，顯示需要一套能整合資訊、協助評估與快速決策的智慧系統。



意見整合
避免討論冗長



預算分析
降低預算落差



路線規劃
提升交通便利

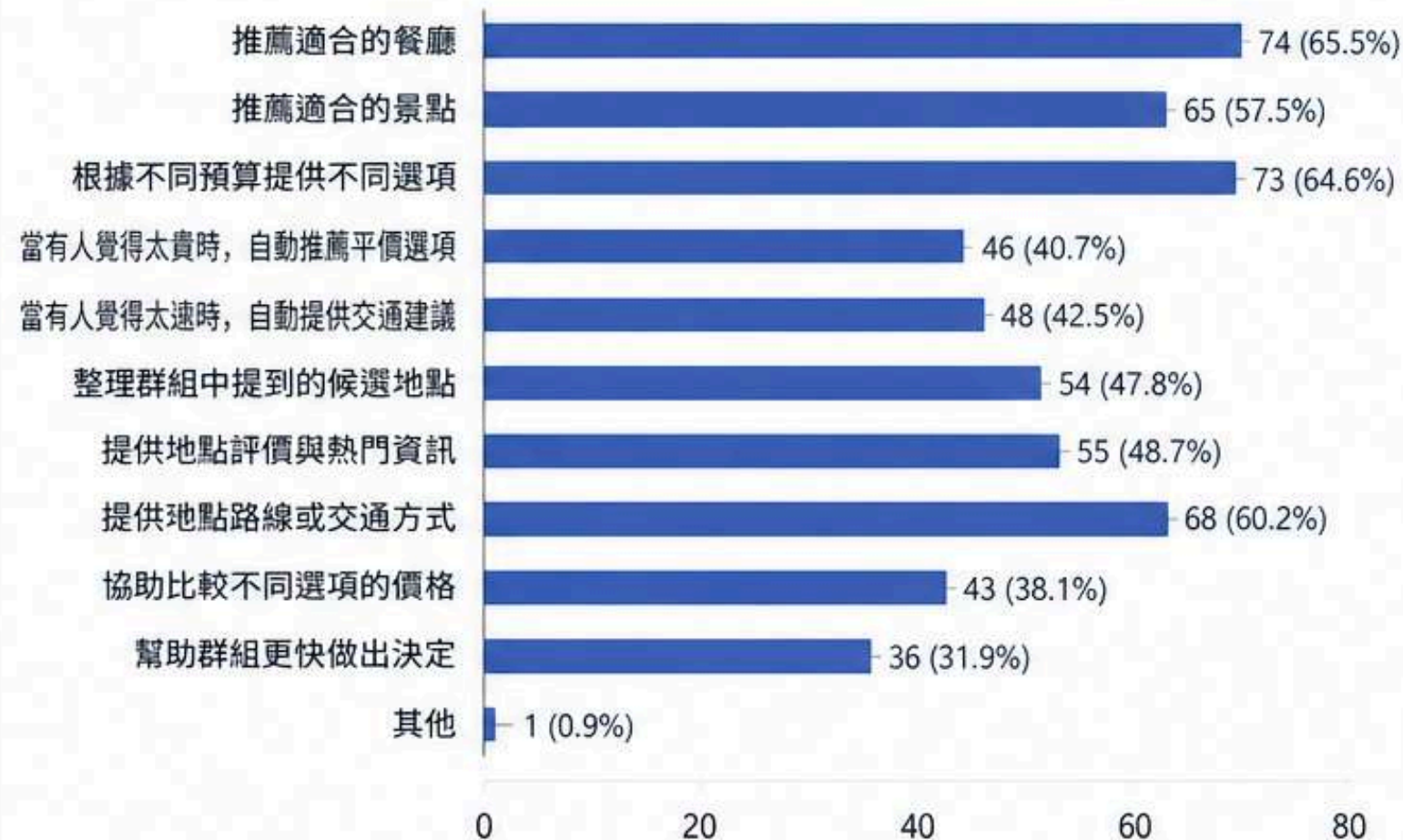


資訊整合
快速查找比較



12. 如果在群組討論過程中，有一套系統能即時協助提供資訊，您希望它具備哪些功能？（可複選）

113 份回應

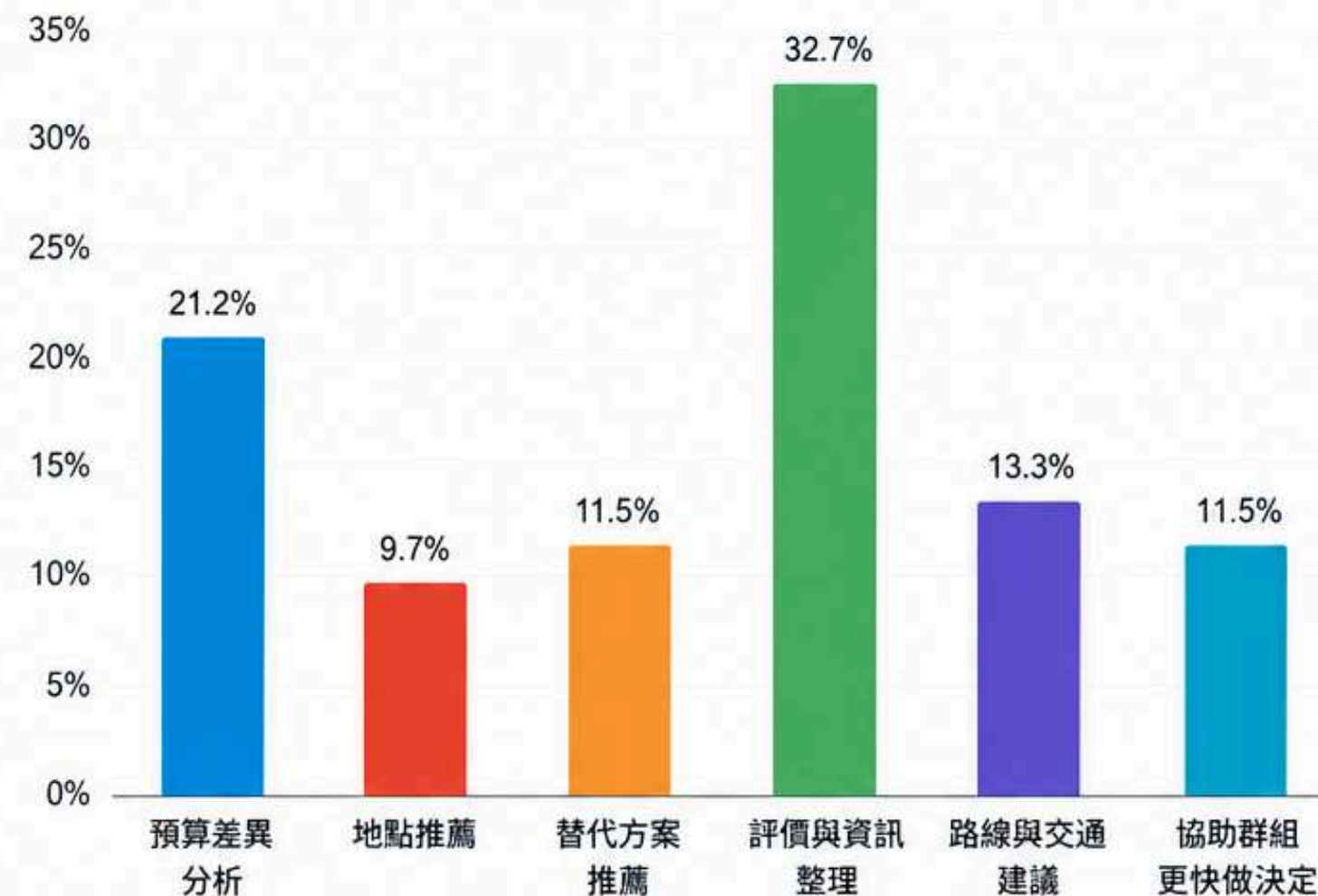


多數使用者希望系統能整合推薦、預算、交通、評價等資訊，並協助比較選項與快速決策。



13. 在以下功能中，您最重視哪一項？

113 份回應



「評價與資訊整理」為最受重視的功能（32.7%），其次為「預算差異分析」（21.2%）與「路線與交通建議」（13.3%）。



分析重點

使用者期望系統能整合多種資訊並提供智慧建議，其中「評價與資訊整理」、「預算分析」與「路線建議」最受重視，顯示現有工具仍缺乏全面整合與即時協助決策的能力。

使用者最在意的功能對應我們的解決方案



預算差異分析

自動比對預算差異
推薦合適選項



地點推薦

依需求與偏好
推薦餐廳與景點



替代方案推薦

提供多種替代方案
增加選擇彈性



評價與資訊整理

彙整評價與熱門資訊
節省查找時間



路線與交通建議

規劃最佳路線
與交通方式



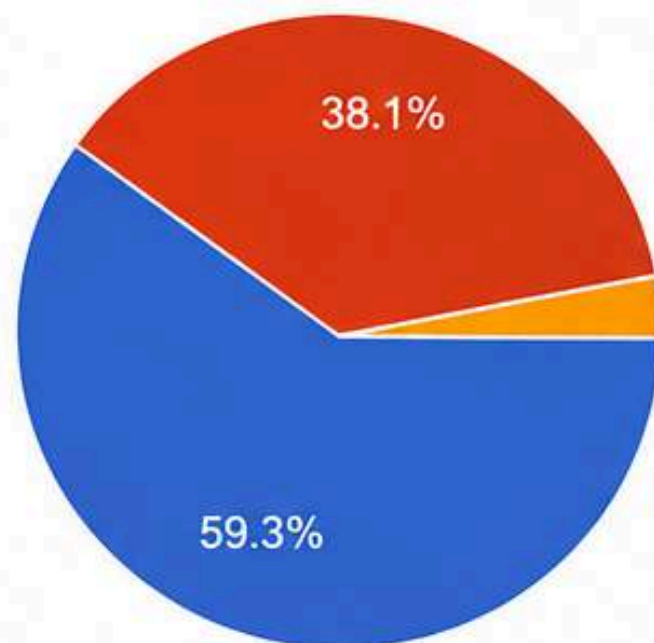
協助群組 更快做決定

整合資訊與投票
提升決策效率

14

如果有一套系統能在群組討論時，協助整合地點、預算與路線資訊，您會願意使用嗎？

113 份回應



非常願意	67 人	(59.3%)
願意	43 人	(38.1%)
普通	3 人	(2.7%)
不太願意	0 人	(0%)
非常不願意	0 人	(0%)

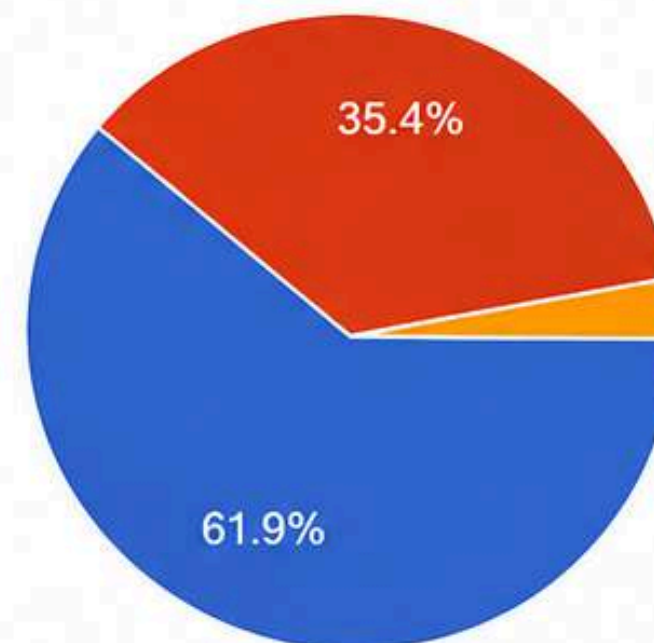


97.4% 的受訪者表示願意使用此系統
(非常願意 + 願意)，顯示高度使用意願。

15

若此系統能減少反覆查詢與整理資訊的時間，您會願意推薦給朋友嗎？

113 份回應



非常願意	70 人	(61.9%)
願意	40 人	(35.4%)
普通	3 人	(2.7%)
不太願意	0 人	(0%)
非常不願意	0 人	(0%)



97.3% 的受訪者表示願意推薦給朋友
(非常願意 + 願意)，顯示高推薦意願與市場潛力。



綜合發現

根據問卷結果，大多數受訪者對於系統的使用與推薦持正面態度，顯示若能整合地點、預算與路線資訊，確實能有效解決群組討論中的痛點，並具備高度的市場需求與實際應用價值。

願意使用比例 (非常願意 + 願意)

97.4%

願意推薦比例 (非常願意 + 願意)

97.3%



重要意義



高使用意願

使用者期待透過系統整合資訊，提升討論效率與決策品質。



高推薦意願

能減少整理與查詢時間，提升體驗，使用者願意推薦給他人使用。

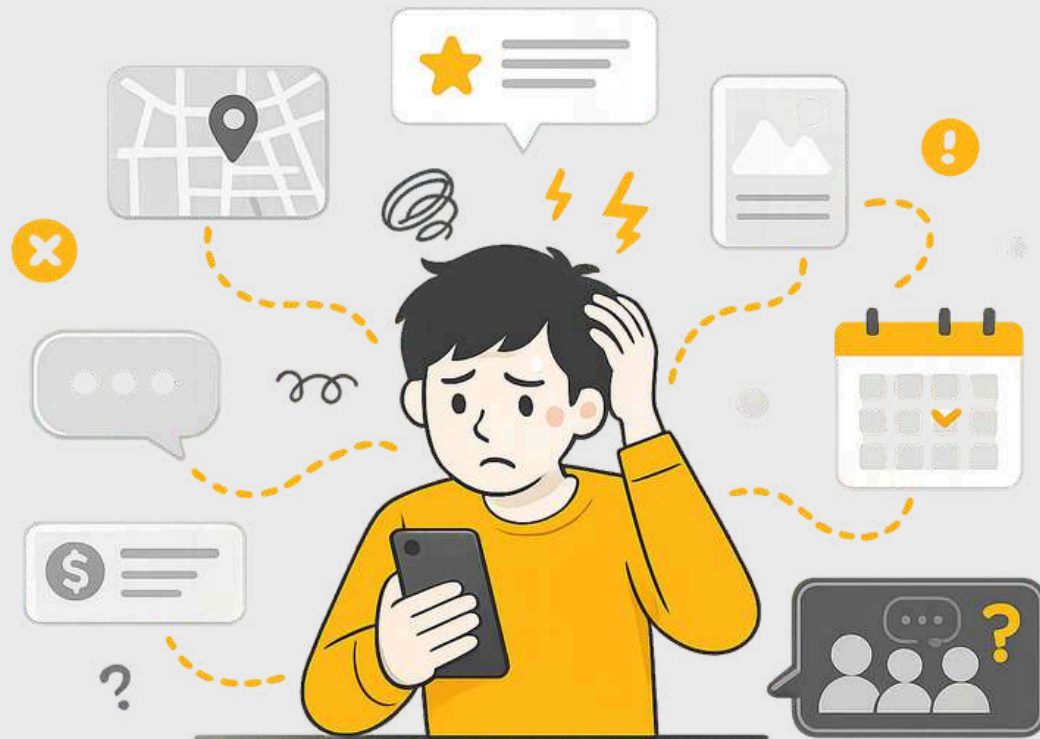


市場需求明確

顯示本研究所提出的系統具備實際需求與發展價值。

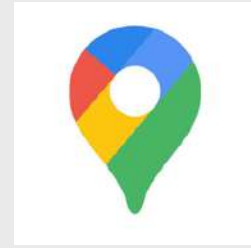
系統整體規劃

現有工具限制



資訊分散，決策效率低

常見工具比較



Google
Maps

- ✗ 手動搜尋
- ✗ 無法分析
群組對話



LINE
旅遊

- ✗ 資訊整合
- ✗ 缺乏決策支援



去趣
chictrip

- ✗ 手動建行程
- ✗ 無法分析
群組對話



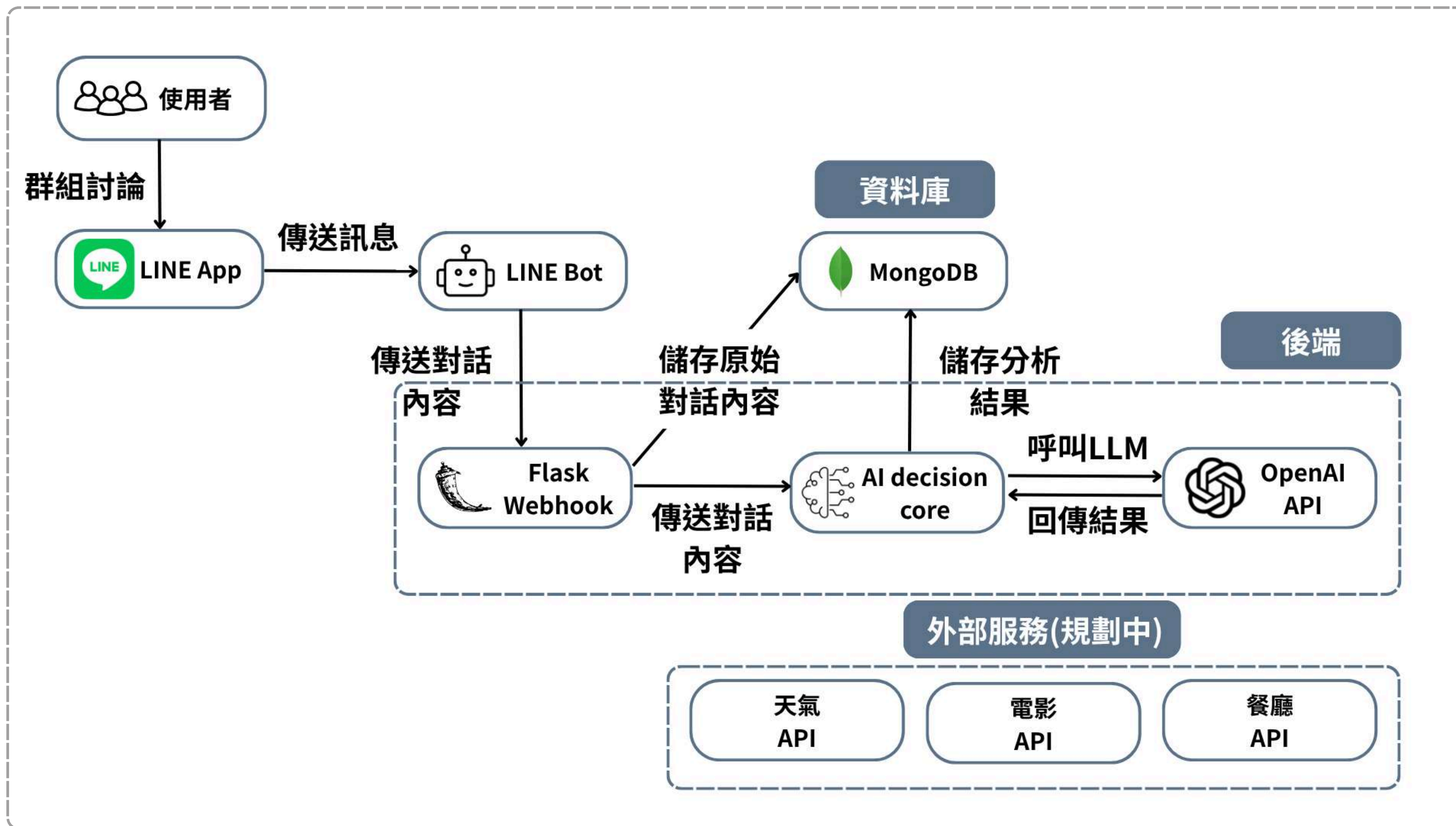
Funliday

- ✗ 行程編輯
- ✗ 缺乏決策輔助



核心問題

現有工具多為**被動搜尋**模式，
缺乏從群組對話中**主動提供決策支援**的能力。



01

LINE 群組情境整合



直接從群組對話理解需求與條件，貼近使用者日常情境。

03

多人決策支援



支援條件整合、意見彙整與共識形成，協助群組快速做出決定。

02

AI 智慧推薦



運用 AI 分析對話內容，自動整理條件並推薦最符合需求的餐廳或景點。

04

不需切換平台



在 LINE 群組內即可完成查詢、推薦與行程規劃，減少跨平台操作負擔。



01



對話內容**辨識**仍可能有**誤差**，
影響條件擷取與推薦結果準確度。

03



依賴**外部資料來源**，
資料更新頻率可能影響推薦品質。

02

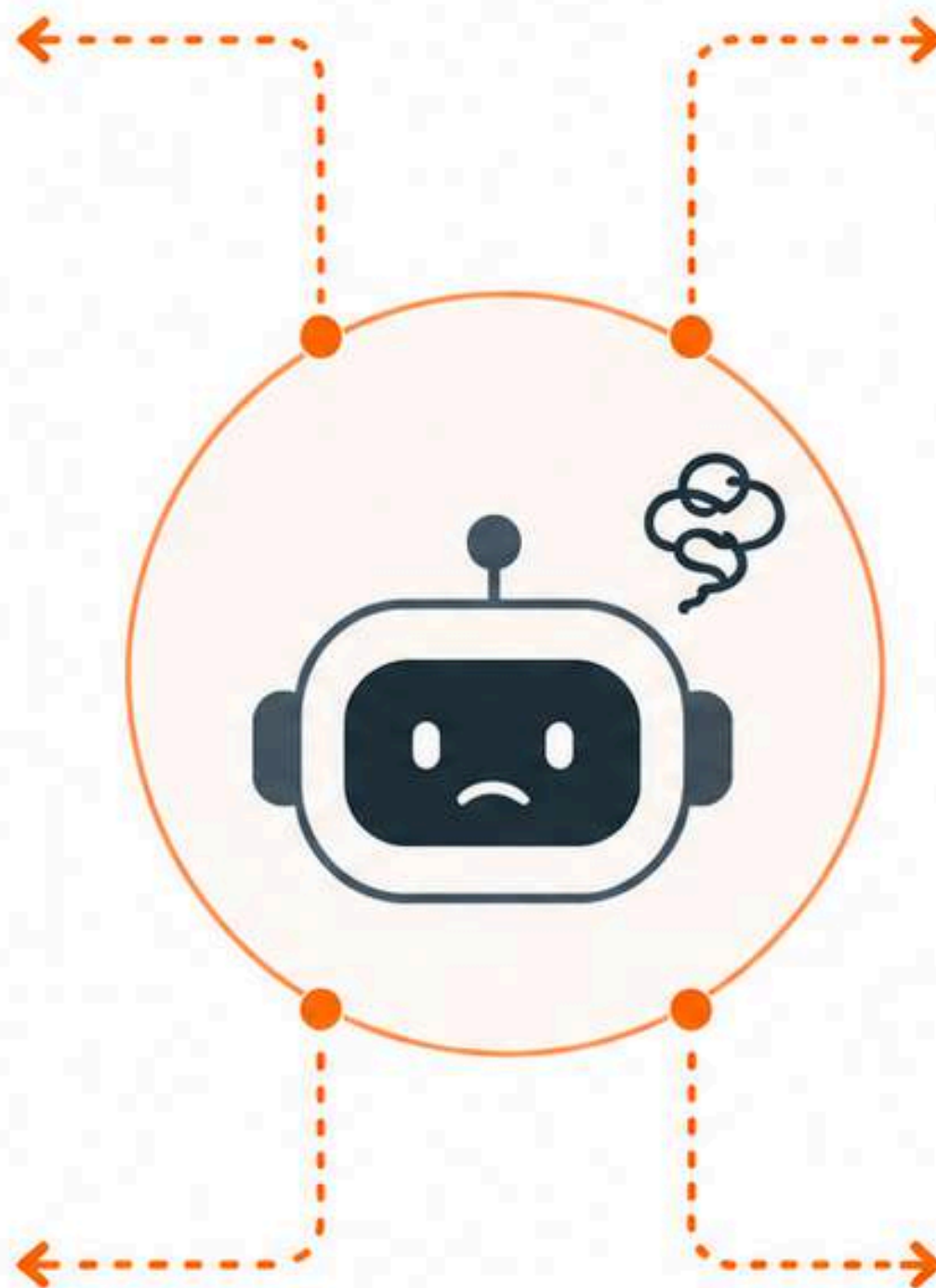


AI 判斷**缺乏使用者真實意圖**，
使用者偏好可能隨情境改變，
AI 難以完全理解個人隱性需求，
可能導致推薦結果與實際期待有所落差。

04



使用者需**培養新的使用習慣**，
初期學習與接受度可能有限。



01



LINE 使用率高

85% 的受訪者最常使用 LINE 群組進行聚餐與出遊討論，遠高於其他溝通方式。

03



聚餐與出遊需求普遍

多人聚餐與出遊為日常高頻需求，市場潛力大。

02



AI Agent 發展快速

AI 技術持續進步，對話理解與推薦能力快速提升。

04



智慧生活整合趨勢

使用者傾向在單一平台完成多元服務，追求便利與效率。



01



既有平台競爭

Google Maps、旅遊平台等功能成熟，市占率高，使用者已形成習慣。

02



AI 技術快速發展

ChatGPT 等 AI Agent 快速進化，可能提供類似甚至更強的服務。

03



資料維護成本高

景點、餐廳、價格等資訊需持續更新，可加資料維護與營運成本。

04



隱私與安全風險

處理使用者對話內容與偏好資訊，需確保隱私保護與資料安全。

05



市場接受度待驗證

使用者需改變既有流程與習慣，市場接受度仍有不確定性。

主要競爭工具



Google Maps

地圖導航
地點搜尋



ChatGPT Agent

AI 助理
任務執行能力

funliday

Funliday 等旅遊平台

行程規劃
景點推薦

VS

本系統優勢



群組情境分析

專注 LINE 群組情境，直接從對話內容理解需求。



多人決策支援

整合條件、意見彙整，協助群組快速達成共識。



一站式整合服務

推薦、查詢、行程規劃集中於同一平台完成。

前端系統



使用者加入「AI 旅遊行程助理」官方帳號，可透過下方選單進入互動式地圖網頁，瀏覽系統推薦之景點、餐廳與行程規劃結果。







需求蒐集與條件整合

從群組對話中整理時間、地點、人數與偏好資訊。

包含劇本

- 劇本一：時間解析與需求啟動
- 劇本二：活動意圖辨識
- 劇本三：需求整合
- 劇本六：行程資訊補全



行程規劃與路線最佳化

依據條件自動產生行程並優化安排。

包含劇本

- 劇本四：自動行程生成
- 劇本五：路線最佳化
- 劇本十四：電影／影城
- 劇本十五：展覽／博物館
- 劇本十六：臨時決定



多人決策與想法整合

協助群組整合想法與完成決定。

包含劇本

- 劇本七：多人決策與衝突處理
- 劇本八：討論停滯與 AI 補位
- 劇本九：投票決策
- 劇本十二：決策完成與執行支援

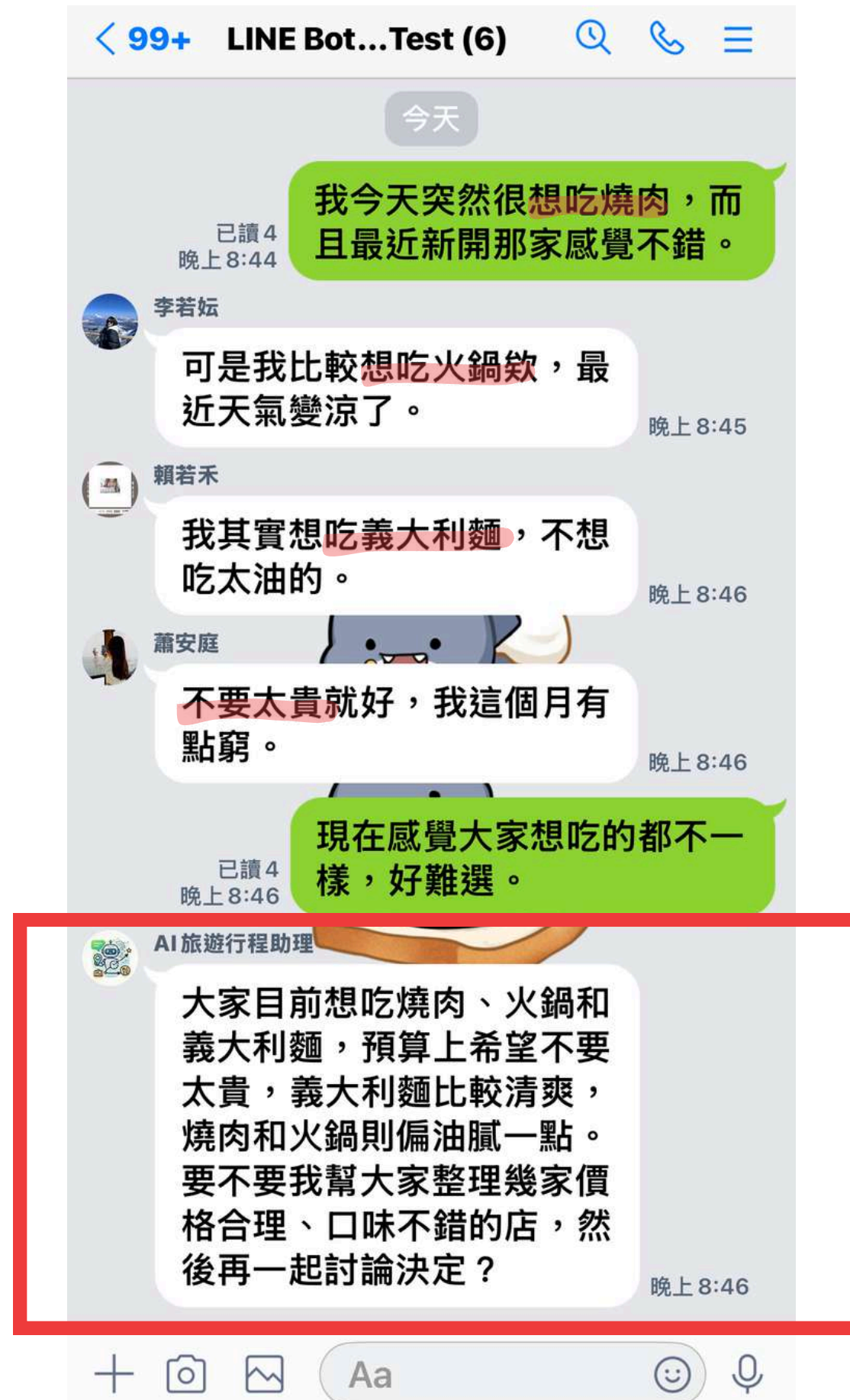


風險評估與情境應變

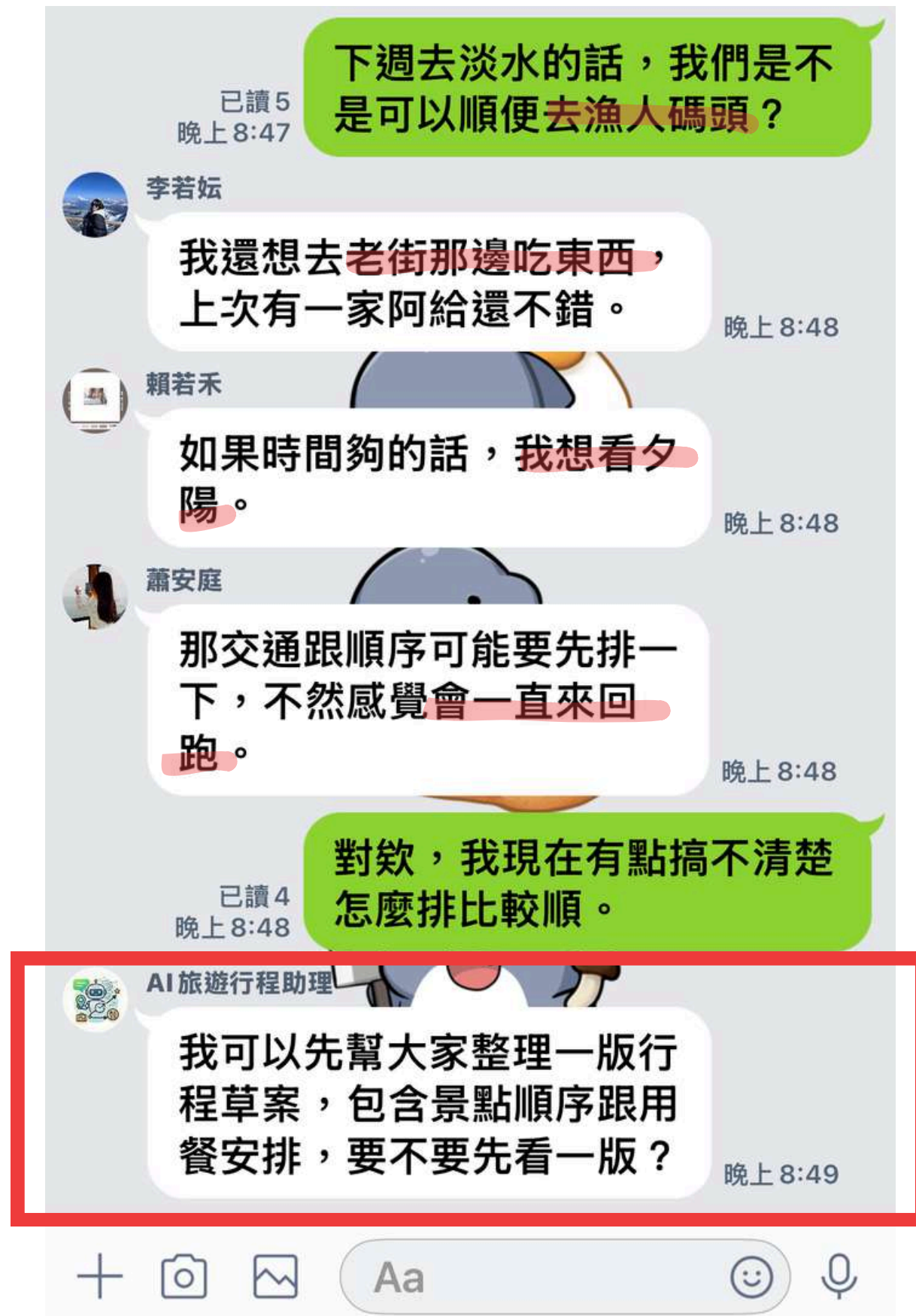
辨識時間、天氣與活動限制等風險因素。

包含劇本

- 劇本十：時間衝突偵測
- 劇本十一：預算與延伸資訊補充
- 劇本十三：音樂劇／演唱會
(時間不可延誤型)
- 劇本十七：戶外活動
(天氣影響型)



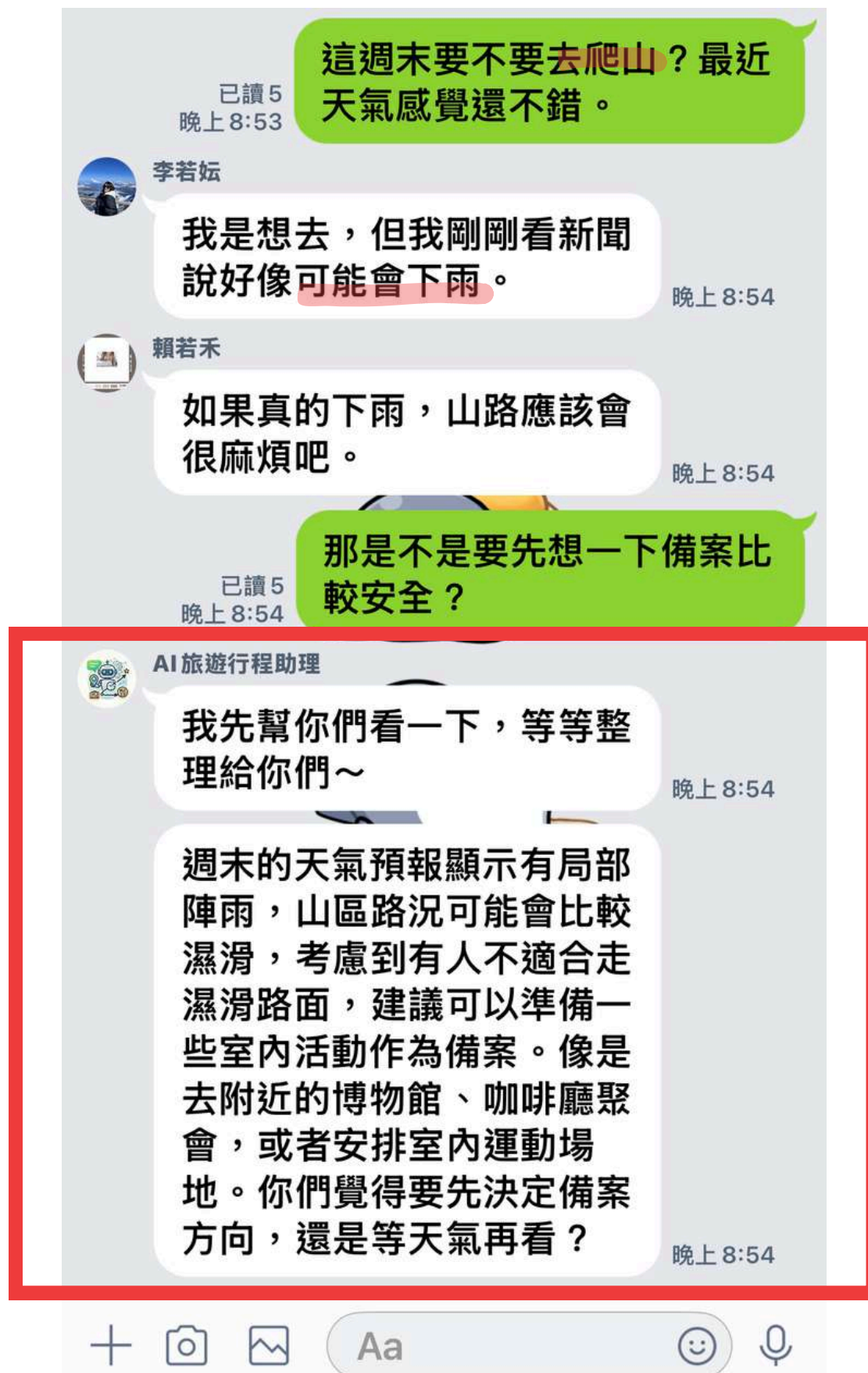
群組成員在聚餐選擇上出現偏好分歧，且同時受到天氣、口味與預算影響時，AI可透過對話分析整理主要需求，協助群組收斂共識並提升決策效率。



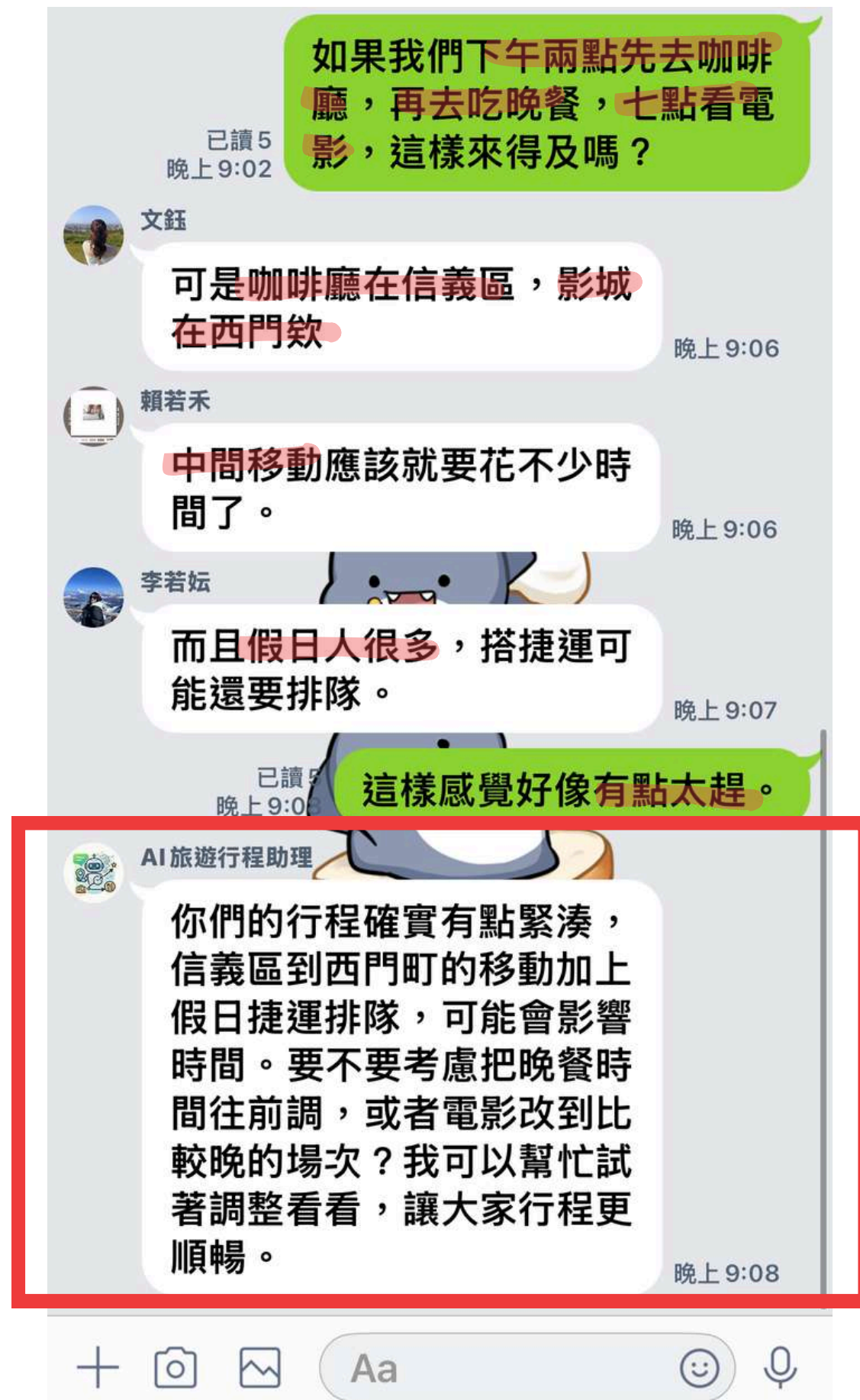
當群組成員對出遊地點已有初步想法，但仍需整合景點順序、用餐安排、時間節點與交通動線時，AI可透過對話分析協助整理需求，並進一步產出初步行程草案，提升規劃效率與清晰度。



當群組成員在休閒活動安排上需同時考量可行時段、場次限制與活動先後順序時，AI可透過對話分析整理主要條件，協助群組快速篩選合適方案並提升決策效率。



當群組成員在戶外活動規劃中遇到天氣不確定因素，並開始考量安全性與替代方案時，AI可透過對話分析整合天氣資訊與活動條件，協助群組提前準備備案並提升決策彈性。



當群組成員規劃的活動地點跨區分散，且可能因交通時間與人潮因素導致行程過於緊湊時，AI可協助辨識動線衝突，並提出較順暢的時間與順序調整建議。

後端系統

AI Decision Core

接收與安全驗證

```
@app.route("/callback", methods=["POST"])
def callback() -> str:
    signature = request.headers["X-Line-Signature"]
    body = request.get_data(as_text=True)
    app.logger.info("Request body: %s", body)

    try:
        handler.handle(body, signature)
    except InvalidSignatureError:
        app.logger.info("Webhook 簽章驗證失敗，請確認 channel access token / secret。")
        abort(400)

    return "OK"
```

上下文劇本分析

```
print(f"DEBUG 送進 AI 的上下文: \n{context_text}")
result_obj = analyze_dialogue(context_text)
result = result_obj.to_dict()
```

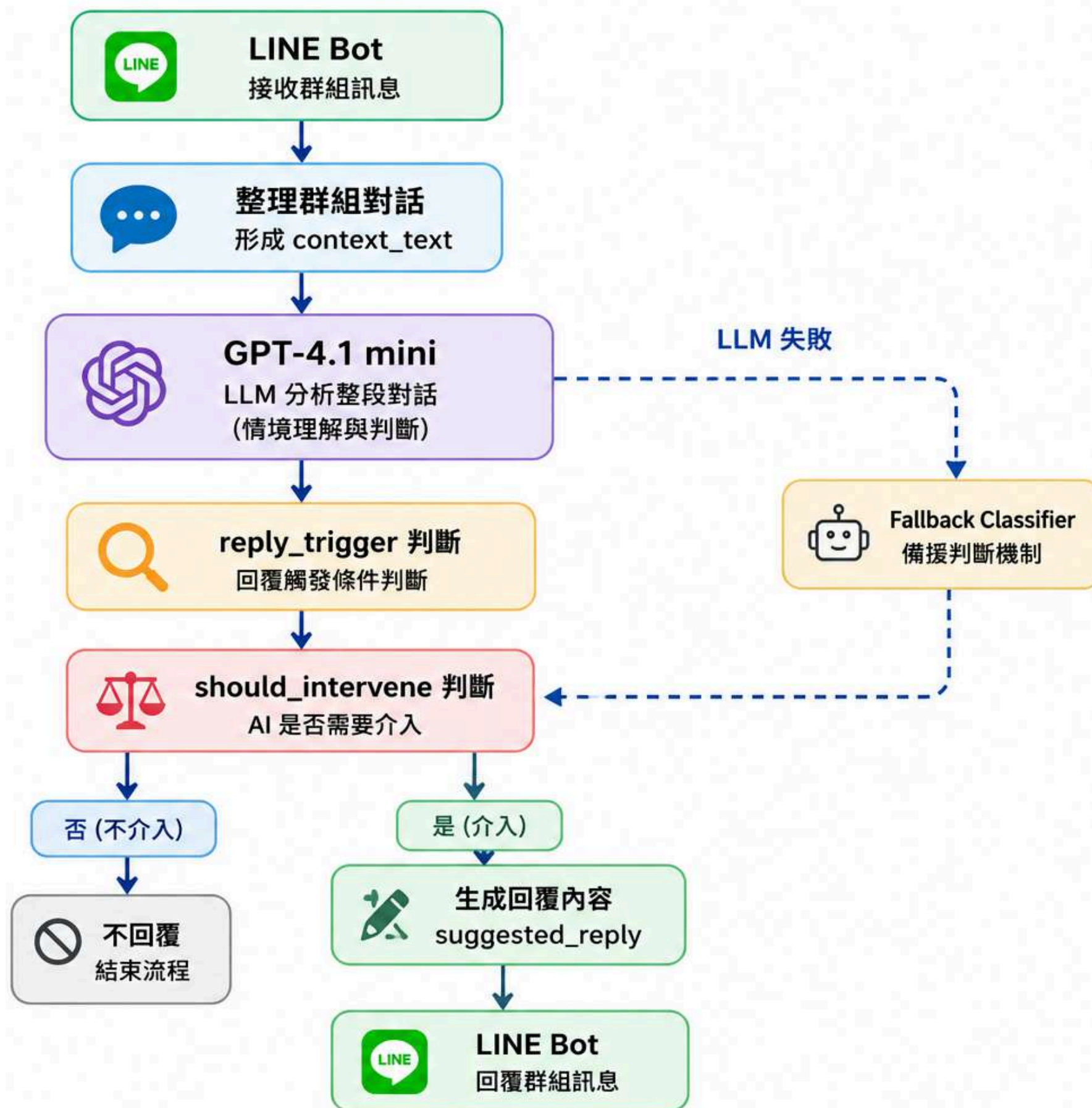
非同步雙軌回覆優化

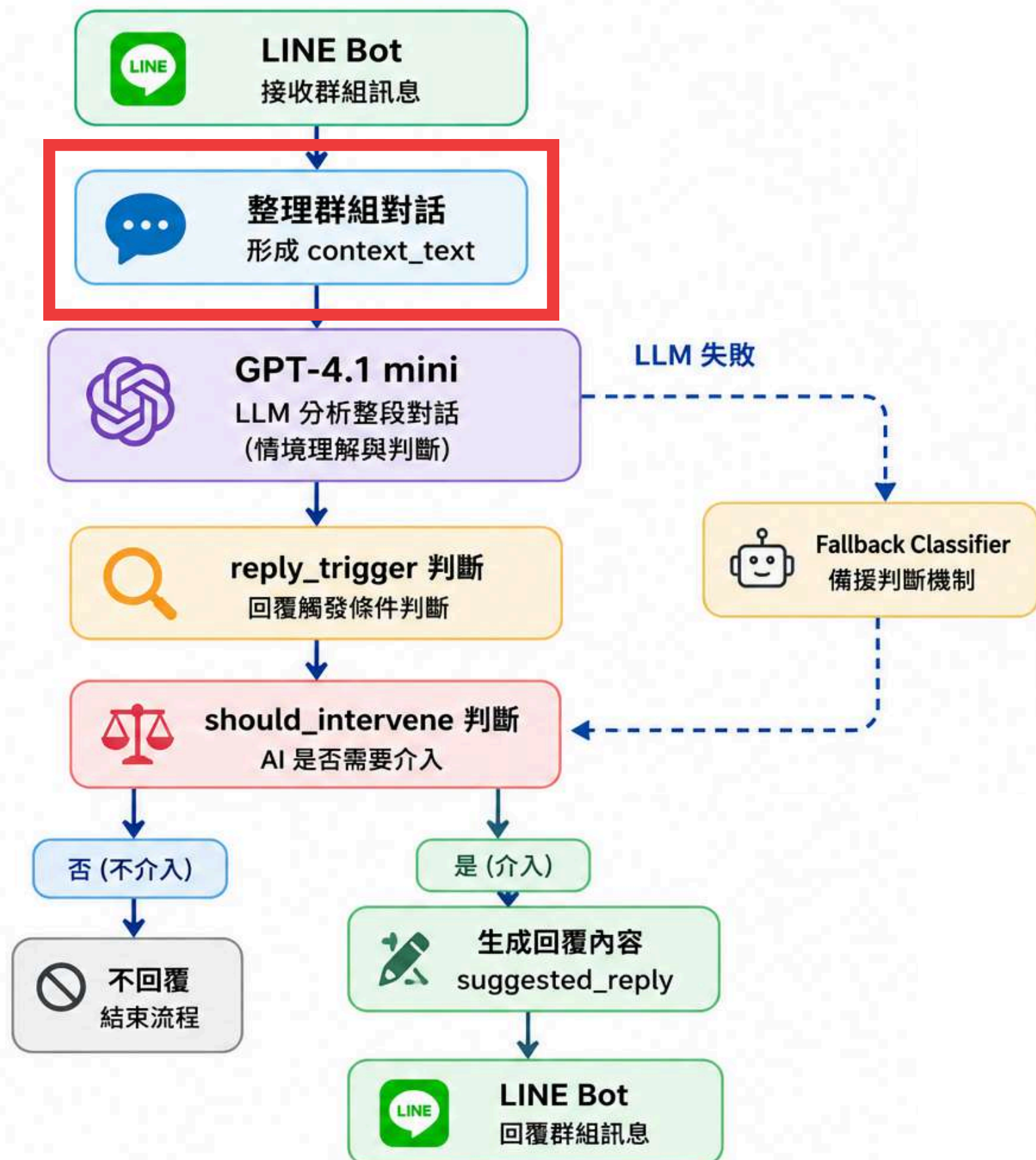
```
if requires_external_search:
    push_target_id = _get_push_target_id(event)
    final_reply = _resolve_final_reply_after_external_search(result)

    if _should_suppress_duplicate_candidates(
        conversation_key,
        scenario_code,
        current_user_message_count,
        intermediate_reply,
        final_reply,
    ):
        print(
            "略過語意相近的重複回覆。 "
            f"(scenario_code={scenario_code})"
        )
        print("=" * 50 + "\n")
        return

    if push_target_id and intermediate_reply:
        _reply_text(line_bot_api, event.reply_token, intermediate_reply)
        _mark_reply_sent(
            conversation_key,
            scenario_code,
            intermediate_reply,
        )
        print(f"先回覆查詢中訊息: {intermediate_reply}")

        threading.Thread(
            target=_push_followup_after_external_search,
            args=(conversation_key, push_target_id, scenario_code, result),
            daemon=True,
        ).start()
```





01 保留最近幾則訊息

```
CONVERSATION_WINDOW_SIZE = max(1, int(os.getenv("CONVERSATION_WINDOW_SIZE", "15")))

@dataclass
class ConversationState:
    history: deque[str] = field(
        default_factory=lambda: deque(maxlen=CONVERSATION_WINDOW_SIZE)
    )
    user_message_count: int = 0
```

02 建立 context_text

新訊息加入對話紀錄

```
normalized_text = user_text.strip()
```

取出所有訊息

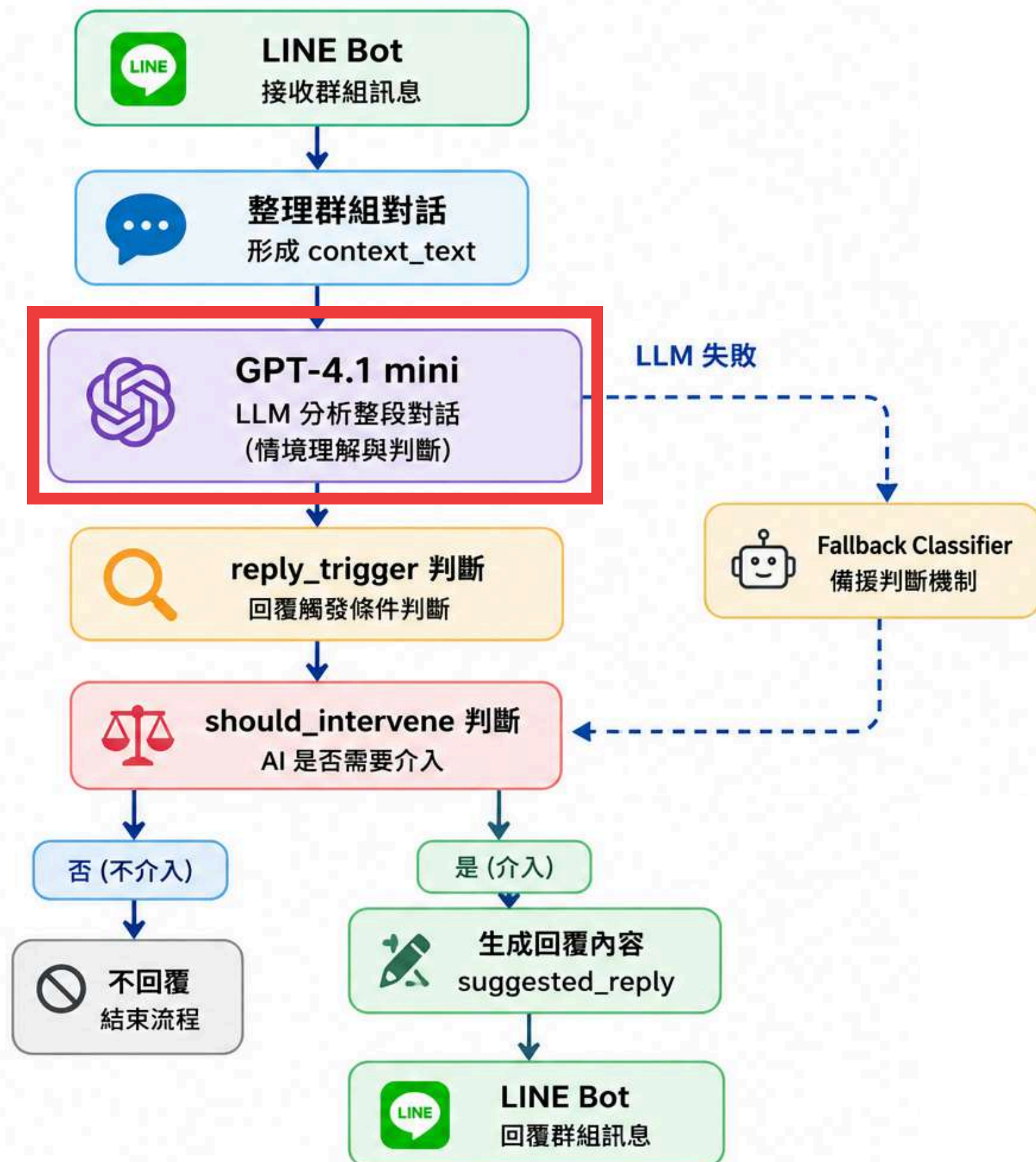
```
history_snapshot = list(state.history)
```

將多則訊息合併成完整對話

```
context_text = "\n".join(history_snapshot)
```

```
SCENARIOS: list[ScenarioDefinition] = [  
    ScenarioDefinition(  
        code="劇本一",  
        name="時間解析與需求啟動",  
        stage="發起階段",  
        should_intervene=False,  
        intervention_type="隱性介入",  
        system_behavior=["抽取時間", "建立初步時間框架", "持續觀察"],  
        suggested_reply="",  
        keywords=["這週六", "週六", "中午", "晚上", "有空", "不要太晚"],  
        feature_hints=["has_time", "not_has_location", "not_planning"],  
    ),
```

```
ScenarioDefinition(  
    code="劇本四",  
    name="自動行程生成",  
    stage="方案產生階段",  
    should_intervene=True,  
    intervention_type="顯性介入",  
    system_behavior=["生成初步行程", "整合景點與餐廳", "提供順序建議"],  
    suggested_reply="我可以先帮大家整理一版行程草案，包含景點順序跟用餐安排，要不要先看一版？",  
    keywords=["排行程", "順序", "景點", "餐廳", "一日行程"],  
    feature_hints=["planning", "has_location"],  
),
```

第一階段

```
judgment_messages = _build_judgment_messages(text, extracted_info)
judgment_data = _call_openai_json(
    client,
    model,
    judgment_messages,
    purpose="情境判斷",
)
judgment_result = _normalize_result(judgment_data, extracted_info)

if not judgment_result.should_intervene:
    return judgment_result
```

判斷scenario_code、reply_trigger、should_intervene

第二階段

```
generation_messages = _build_generation_messages(text, judgment_result)
generated_reply = _call_openai_json(
    client,
    model,
    generation_messages,
    purpose="回覆生成",
)
return _merge_generated_reply(judgment_result, generated_reply)
```

需要介入時才執行



Prompt

```
system_prompt = f"""
你是「LINE 群組行程助理」的情境判斷核心模組。
你的工作是根據整段群組對話、對話進展、多人互動方式與摘要資訊，
判斷目前最符合哪一個劇本，以及 AI 是否需要介入。

這一階段只負責「判斷」，不負責最終回覆生成。
所以在這一階段：
- intermediate_reply 一律輸出空字串
- suggested_reply 一律輸出空字串

```

reply_trigger分類

reply_trigger 必須是以下其中一種：

- explicit_request：使用者明確向 AI 求助、要求幫忙、要求整理、要求推薦
- functional_question：使用者提出具有功能性的問題，例如查詢、推薦、規劃、比較、排序
- stuck_discussion：群組討論明顯卡住，成員反覆出現「都可以」、「隨便」、「沒意見」、「你們決定」等附和語句，且沒有新增具體選項、條件或決策方向，對話仍無法推進時
- no_reply：一般聊天、寒暄、附和、閒聊、情緒反應，或尚未形成明確需求時



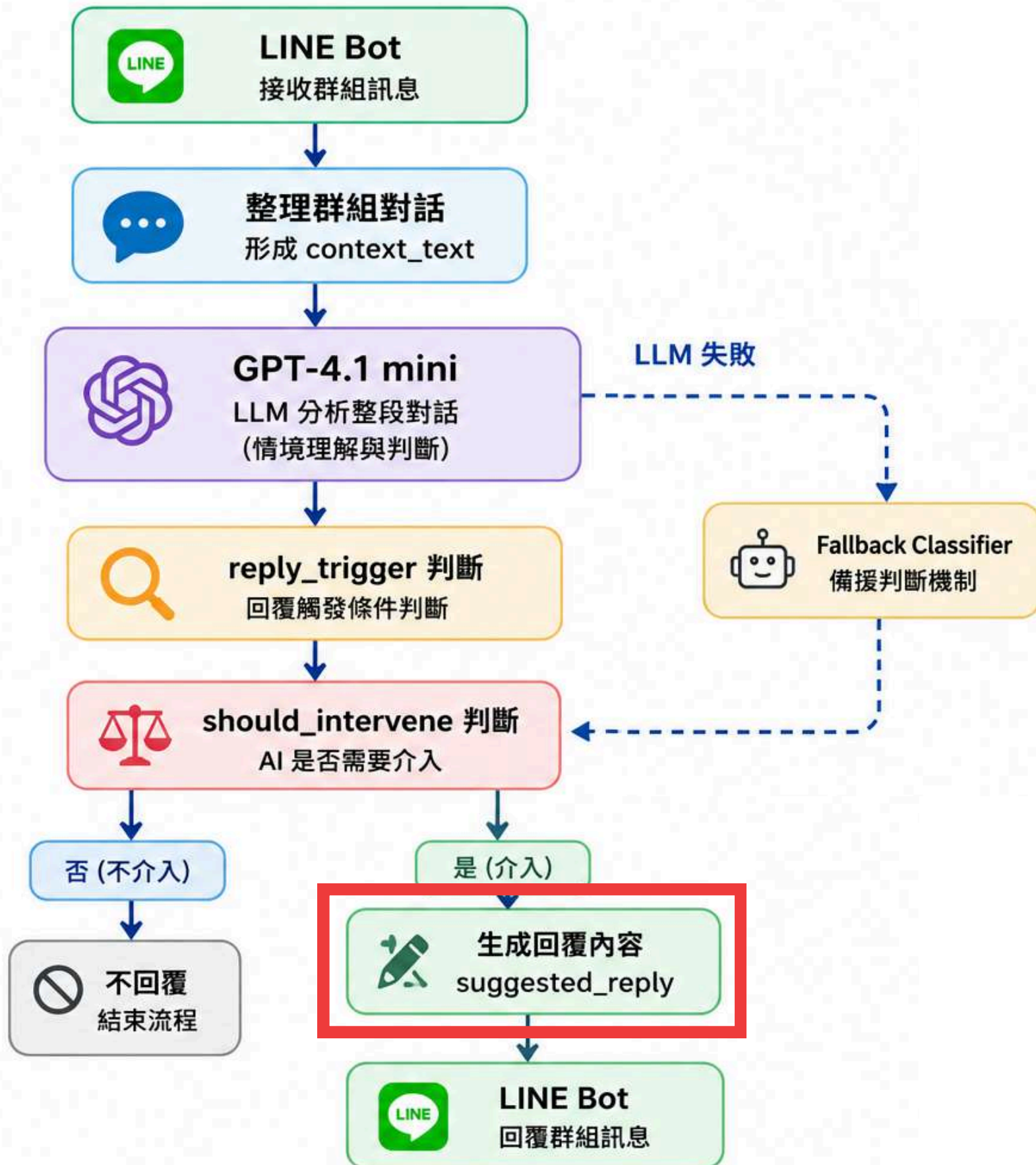
Prompt

重要規則：

- 如果 `reply_trigger = no_reply`，則 `should_intervene` 必須為 `false`。
- 如果 `reply_trigger` 是 `explicit_request`、`functional_question` 或 `stuck_discussion`，則 `should_intervene` 必須為 `true`。
- 如果 `reply_trigger` 是 `explicit_request`、`functional_question` 或 `stuck_discussion`，就不應選擇原本 `should_intervene = false` 的劇本。

AI是否主動介入

```
if reply_trigger == "no_reply":
    should_intervene = False
elif reply_trigger in {"explicit_request", "functional_question", "stuck_discussion"}:
    should_intervene = True
```



生成最後回覆內容

```
generation_messages = _build_generation_messages(text, judgment_result)
generated_reply = _call_openai_json(
    client,
    model,
    generation_messages,
    purpose="回覆生成",
)
return _merge_generated_reply(judgment_result, generated_reply)
```

系統最後正式回給 LINE 群組的建議內容，
只在 `should_intervene = true` 時生成

資料庫

一、核心資料（群組與使用者）

groups（群組資料）		
 _id	OBJECTID	PK
line_group_id	VARCHAR(128)	UNIQUE
group_name	VARCHAR(255)	
created_at	DATETIME	DEFAULT NOW
INDEX (line_group_id)		

二、訊息與摘要（對話資料與 AI 摘要）

messages（群組訊息）		
 _id	OBJECTID	PK
line_message_id	VARCHAR(128)	UNIQUE
group_id	OBJECTID	FK
user_id	VARCHAR(128)	
message_text	TEXT	
sent_at	DATETIME	NOT NULL
INDEX (group_id, sent_at)		

summaries（對話摘要與 AI 分析結果）		
 _id	OBJECTID	PK
group_id	OBJECTID	FK
window_start	DATETIME	
window_end	DATETIME	
summary_text	TEXT	
ai_scenario_code	VARCHAR(50)	
ai_scenario_name	VARCHAR(255)	
ai_should_intervene	TINYINT(1)	
ai_suggested_reply	TEXT	
created_at	DATETIME	DEFAULT NOW
INDEX (group_id, window_start)		

三、AI 結果與行程建議

itineraries（旅遊行程）		
 _id	OBJECTID	PK
summary_id	OBJECTID	FK
group_id	OBJECTID	FK
itinerary_text	TEXT	
created_at	DATETIME	DEFAULT NOW
INDEX (group_id, created_at)		

四、投票與互動（決策輔助）

vote_sessions（投票紀錄）		
 _id	OBJECTID	PK
group_id	OBJECTID	FK
title	VARCHAR(255)	
session_type	ENUM('行程', '景點', '餐廳', '交通', '活動', '其他')	
status	ENUM('進行中', '已結束')	
created_at	DATETIME	DEFAULT NOW
closed_at	DATETIME	
INDEX (group_id, status)		

五、使用者偏好（個人化設定）

user_preferences（使用者偏好）		
 _id	OBJECTID	PK
group_id	OBJECTID	FK
user_id	VARCHAR(128)	
preference_type	ENUM('飲食', '預算', '預算上限', '交通', '住宿', '活動', '其他')	
preference_value	TEXT	
updated_at	DATETIME	DEFAULT NOW
INDEX (group_id, user_id, preference_type)		

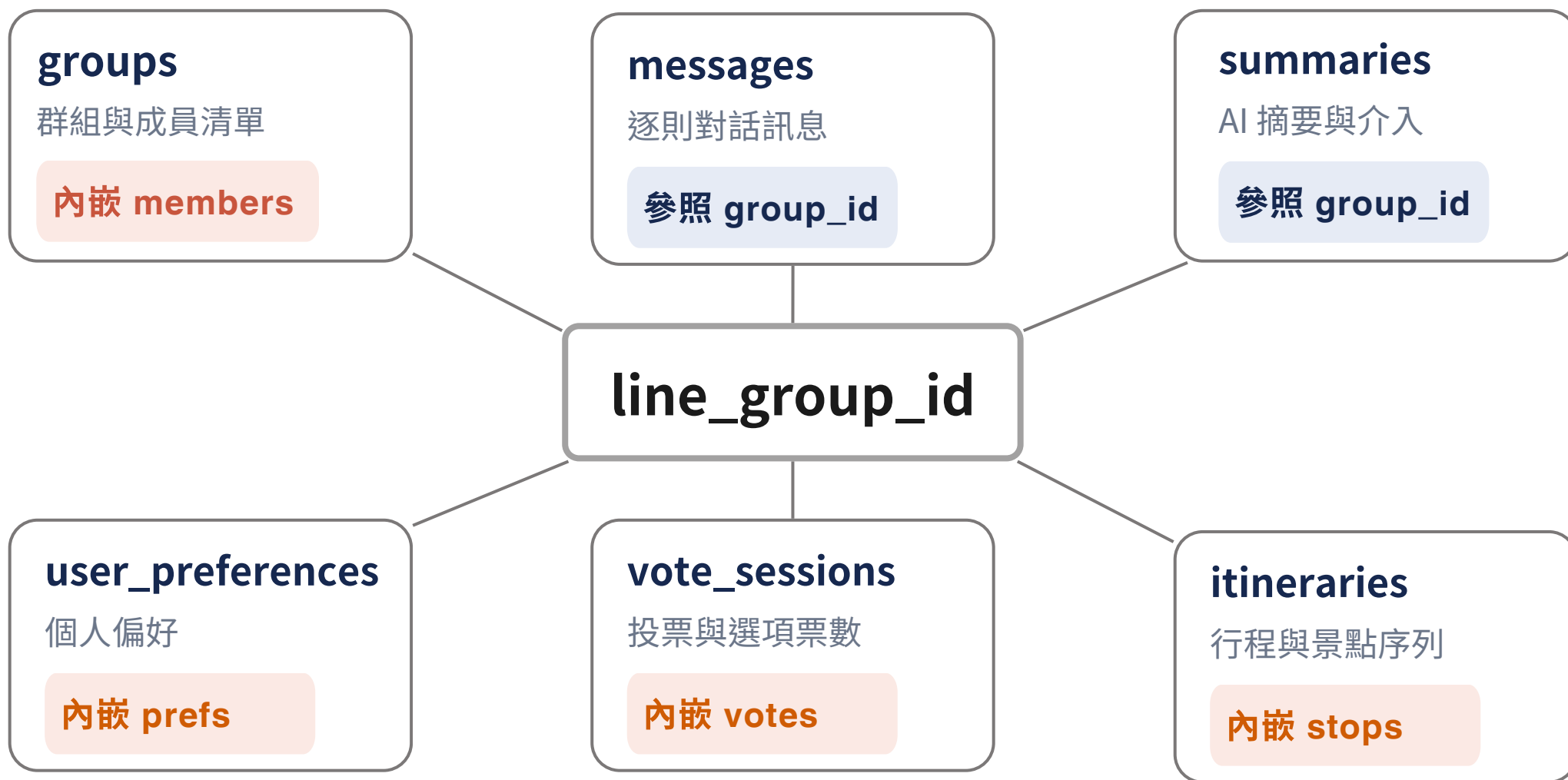
圖例說明

-  主鍵 (Primary Key)
- FK 外鍵 (Foreign Key)
- 1 一對一關係
- N 一對多關係

設計重點

- 採用 MongoDB 文件模型，適合彈性欄位與 JSON 結構資料
- 以群組 (groups) 為核心，串聯訊息、摘要、行程、投票與偏好
- 整合 AI 分析結果，提供情境判斷與建議回覆
- 建立適當索引 (INDEX) 提升查詢效能

6 個 collection，由 line_group_id 串起



itineraries — 內嵌景點序列

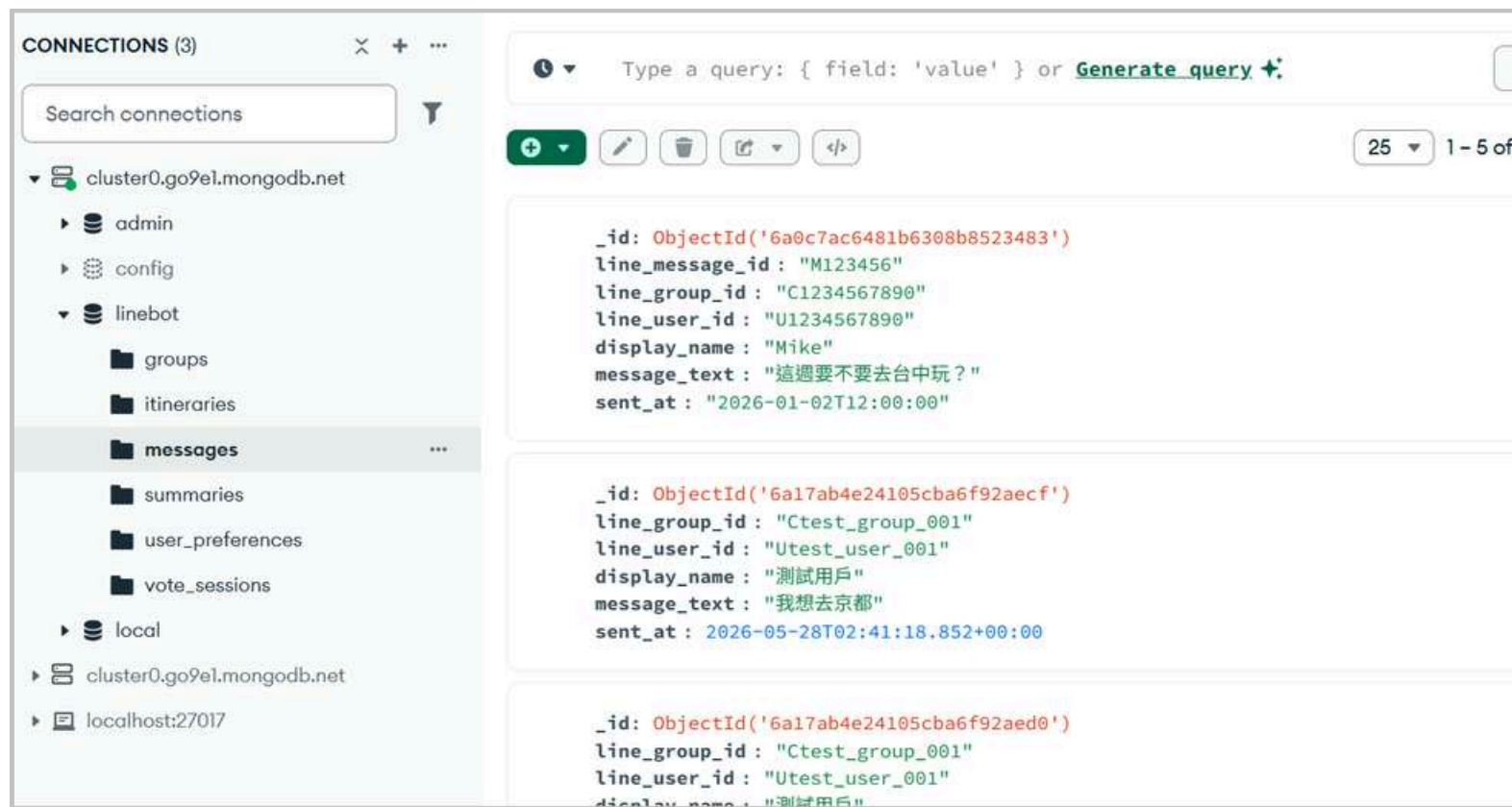
cluster0.go9el.mongodb.net

- admin
- config
- linebot
 - groups
 - itineraries**
 - messages
 - summaries
 - user_preferences
 - vote_sessions
- local

```
{
  "_id": ObjectId('6a0c7ac1481b6308b8523480'),
  "line_group_id": "C1234567890",
  "itinerary_text": "三天兩夜台中行程建議",
  "created_at": "2026-01-02T14:00:00",
  "stops": Array (1)
    0: Object
      stop_order: 1
      stop_name: "高美濕地"
      stop_address: "台中市清水區"
      arrive_time: "10:00"
      duration_min: 120
      category: "景點"
  }
}
```

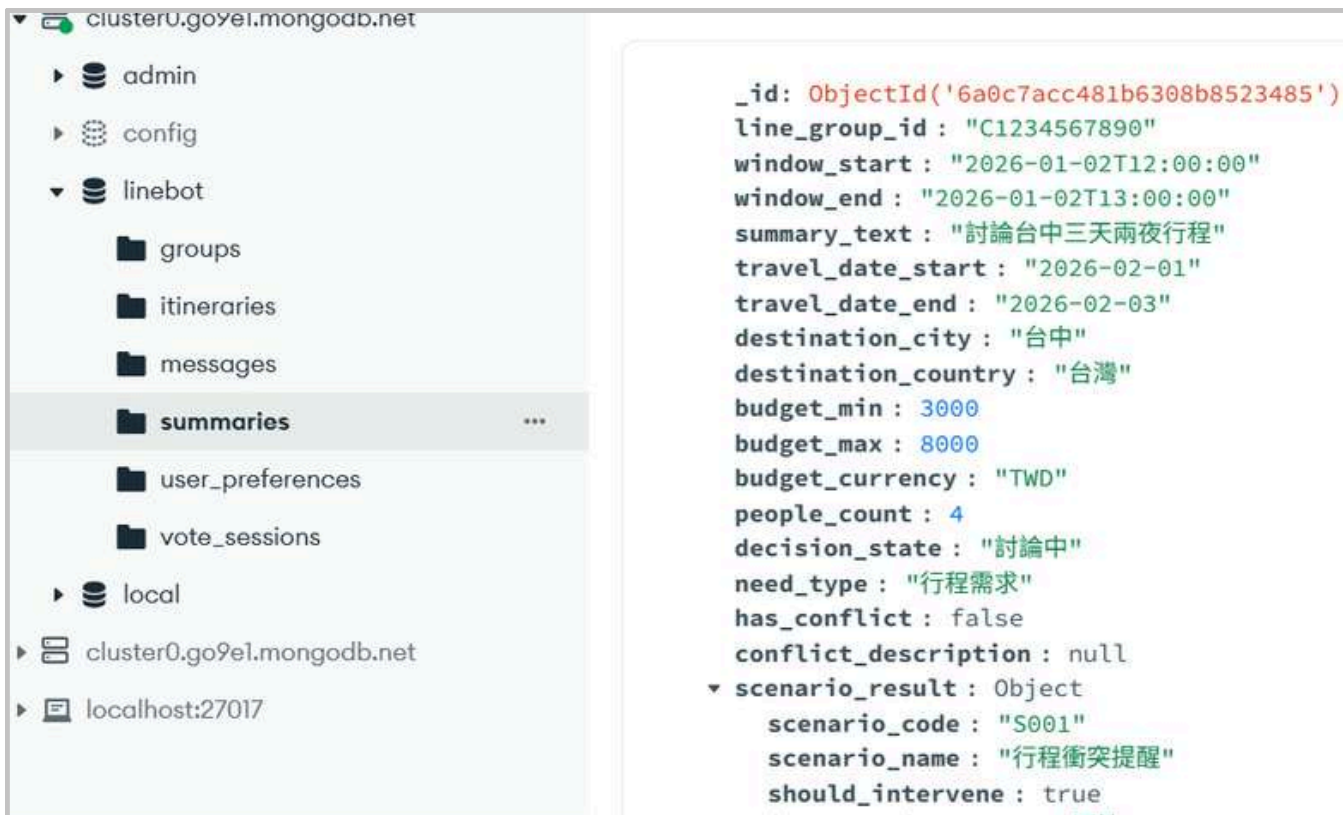
行程展示文，
含時間、停留時長、分類

messages collection



每一筆就是一則訊息，
保留發話者、原文跟時間

summaries — AI 抽取結果

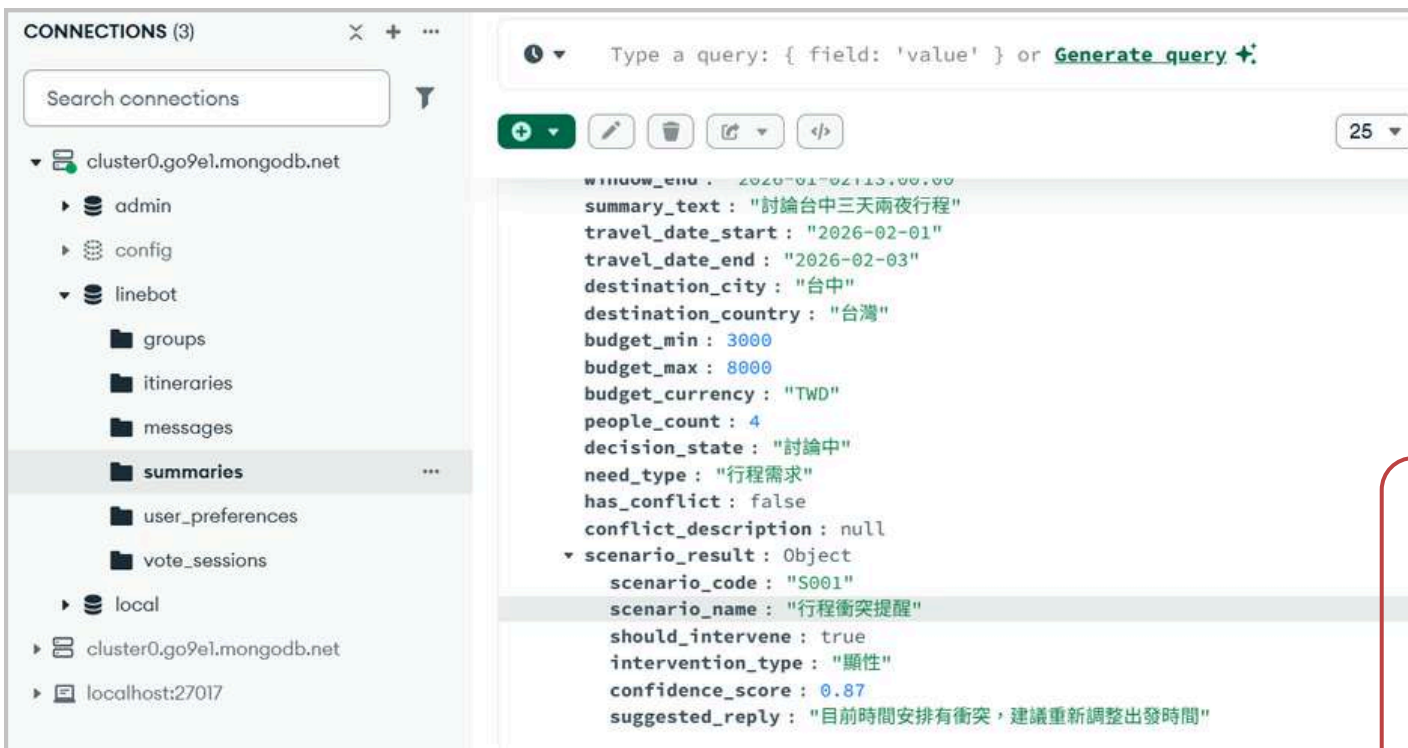


The screenshot shows the MongoDB Compass interface. On the left, the database structure is displayed with the following collections: admin, config, linebot, groups, itineraries, messages, summaries (highlighted), user_preferences, and vote_sessions. Below these are cluster0.go9e1.mongodb.net and localhost:27017. The main panel shows a document from the 'summaries' collection with the following fields:

```
_id: ObjectId('6a0c7acc481b6308b8523485')
line_group_id: "C1234567890"
window_start: "2026-01-02T12:00:00"
window_end: "2026-01-02T13:00:00"
summary_text: "討論台中三天兩夜行程"
travel_date_start: "2026-02-01"
travel_date_end: "2026-02-03"
destination_city: "台中"
destination_country: "台灣"
budget_min: 3000
budget_max: 8000
budget_currency: "TWD"
people_count: 4
decision_state: "討論中"
need_type: "行程需求"
has_conflict: false
conflict_description: null
scenario_result: Object
  scenario_code: "S001"
  scenario_name: "行程衝突提醒"
  should_intervene: true
```

重點放在展開的scenario_result

本系統亮點：scenario_result 介入判斷



行程衝突提醒

should_intervene	true
intervention_type	顯性
confidence_score	0.87

SUGGESTED_REPLY

「目前高美濕地排在上午 10 點，
但大家想看夕陽，
建議改到下午 16:30 前後再前往。」

vote_sessions — 內嵌投票



The screenshot shows the MongoDB Compass interface. On the left, the database structure is displayed with the following hierarchy:

- cluster0.go9e1.mongodb.net
 - admin
 - config
 - linebot
 - groups
 - itineraries
 - messages
 - summaries
 - user_preferences
 - vote_sessions**
 - local
- cluster0.go9e1.mongodb.net
- localhost:27017

The main panel displays the JSON structure of a document in the `vote_sessions` collection:

```
{
  "_id": ObjectId('6a0c7ad6481b6308b8523489'),
  "line_group_id": "C1234567890",
  "title": "要去哪個景點？",
  "status": "進行中",
  "created_at": "2026-01-02T15:00:00",
  "closed_at": null,
  "options": Array (2)
    0: Object
      option_id: 1
      option_text: "高美濕地"
      votes: Array (1)
        0: Object
          line_user_id: "U1234567890"
          voted_at: "2026-01-02T15:10:00"
    1: Object
      option_id: 2
      option_text: "彩虹眷村"
      votes: Array (empty)
}
```

一個投票就是一份文件，
選項用陣列內嵌，
每個選項底下再內嵌投票的人。

Q1 某群組需要 AI 介入的對話

```
db.summaries.find({  
  line_group_id: "C1234567890", has_conflict: true })
```

Q2 統計某投票各選項得票數

```
db.vote_sessions.aggregate([ { $unwind: "$options" },  
  { $project: { option_text:1, n: { $size: "$options.votes" }}} ])
```

Q3 某群組所有不吃辣的成員偏好

```
db.user_preferences.find({  
  "preferences.value": "不吃辣" })
```

商業模式與未來發展

商業模式

以 AI 智慧行程規劃為核心，透過訂閱制、合作分潤與商業媒合建立營收來源，預估每月可達 15 萬營收，於約 2,000 個活躍群組規模下可達成收支平衡。

01

合作夥伴分潤

與餐廳、景點與住宿業者合作，透過推薦導流分潤。

02

廣告與推播收益

結合精準推播與在地廣告，創造額外收益來源。

03

企業與團體方案

提供企業差旅、教育旅遊等客製化行程規劃服務。



1. 群組需求提出

使用者在群組中提出聚餐 / 出遊需求



2. 系統推薦

LINE Bot 推薦符合條件的餐廳 / 景點 / 住宿選項



3. 點擊導流

使用者點擊推薦結果，進入合作夥伴的頁面 (如菜單 / 訂位 / 訂房頁面)



4. 完成預訂 / 消費

使用者完成訂位或消費 (如訂位成功、入住或購票)

合作夥伴 (餐廳 / 景點 / 住宿業者)



餐廳業者



- 提供菜單資訊
- 訂位服務
- 專屬優惠方案



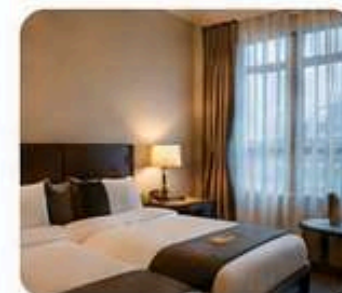
景點業者



- 門票資訊
- 活動資訊
- 優惠票券



住宿業者



- 房型資訊
- 訂房服務
- 專屬優惠

透過 LINE Bot 推薦導流，增加曝光與訂單機會



5. 分潤回饋機制

依合作約定，合作夥伴提供分潤回饋



系統平台
(LINE Bot)



合作夥伴
(餐廳 / 景點 / 住宿業者)

分潤計算方式 (範例)



訂單金額百分比
(如：10%)



固定金額回饋
(如：每筆回饋 \$30)



專屬活動回饋
(如：加碼分潤)

共同創造價值，提升曝光、訂單與營收

好友群組 (6)

A 小明
週六晚上要不要吃飯？ 19:00

B 小美
信義區有人想去嗎？ 19:01

C 阿凱
不要太貴哈哈 19:01

LINE Bot
根據你們的討論，
推薦附近優惠與活動 📌 19:02

為你推薦的優惠與活動

-  **鍋雙涮涮鍋 信義店**
週六限定！四人同行
套餐 85 折 **限時優惠**
到 6/15
-  **信義威秀影城**
指定電影票 75 折
(週末適用) **限時優惠**
到 6/15
-  **信義週末市集**
6/14 - 6/15 美食、文創、
音樂表演 **在地活動**
6/14-6/15
-  **清心福全 信義店**
指定飲品
買一送一 **限時優惠**
到 6/14

AI 根據情境插入廣告



在聊天中 AI 插入的廣告 / 優惠內容

在地餐廳優惠

信義區人氣餐廳
套餐 9 折優惠券
[查看優惠](#)

限時折扣券

電影票優惠券
75 折
週末限定
威秀影城電影票
週末 75 折券
[立即領取](#)

周邊活動曝光

信義週末市集活動
6/14 - 6/15 舉辦
[了解更多](#)

廣告版位

CITIZEN AD
人氣腕錶新品上市
經典系列 9 折起
[立即查看](#)

贊助內容

AD
夏日新品登場！
限量買一送一
[了解更多](#)

品牌曝光

Kiehl's AD
夏日保濕推薦
滿\$2000現折\$200
[立即選購](#)

※ 廣告 / 優惠會依情境動態調整

曝光收益模式（即使未成交，也能靠曝光賺錢）

 **廣告曝光收益**
使用者看到廣告，
平台即可獲得曝光收益
(按曝光次數計費)

 **點擊推播收益**
使用者點擊優惠或廣告，
平台可獲得點擊收益
(按點擊次數計費)

 **品牌贊助版位**
商家贊助固定版位，
品牌曝光提升知名度
(按檔期 / 版位收費)

 即使未完成消費，平台仍可透過廣告曝光創造收益！

< 公司旅行討論群 (12)

A 小明

我們部門想辦兩天一夜員工旅遊，地點希望在宜蘭或花蓮，人數大約20人，預算每人\$5,000內，有推薦行程嗎？

10:30

B 人資-小美

想要包含交通、住宿、景點和餐飲，活動可以輕鬆一點~

10:31

LINE Bot

已為您量身打造企業員工旅遊方案！

✓ 符合預算與人數

✓ 包含交通、住宿、餐飲與活動

✓ 彈性調整行程內容

👉 點擊查看專屬行程方案

10:32

企業專屬行程方案

宜蘭兩天一夜員工旅遊



👤 20人 | 2天1夜 | 每人 \$4,800

方案特色

• 舒適住宿：宜蘭礁溪溫泉飯店

• 在地美食：特色餐食安排

• 團體活動：手作DIY體驗

• 專車接送：台北出發往返

查看完整行程表

+ 📷 🖼️ Aa 😊 🎤



適用對象



企業團隊
員工旅遊 / 團建活動



教育機構
校外教學 / 畢業旅行



社團組織
社團出遊 / 活動旅遊



政府機關
考察參訪 / 公務出行

服務內容



交通安排
專車 / 高鐵 / 機票



住宿規劃
飯店 / 民宿 / 團體房



景點與活動
景點門票 / 體驗活動



餐飲安排
團餐 / 特色餐廳



保險與支援
旅遊保險 / 緊急支援

方案優勢

✓ 客製化行程：依預算、需求與喜好量身打造

✓ 彈性調整：行程內容可彈性調整與優化

✓ 專人服務：專屬客服全程協助與支援

✓ 省時省力：一站式整合規劃，輕鬆成團出發

合作方案

標準方案

適合中小型團體

\$4,500 起 / 人
(20人以上)

進階方案

豐富行程與活動體驗

\$6,000 起 / 人
(20人以上)

 尊榮方案

高品質客製化服務

\$8,000 起 / 人
(20人以上)

 企業專屬服務，讓每一次團體出遊都更輕鬆、更有價值！

未來發展

持續優化智慧推薦與決策能力，拓展更多生活情境應用，打造更完整的智慧旅遊體驗。

01

智慧行程規劃與最佳化

自動規劃最佳路線、停留時間與交通安排，產生完整且個人化的行程規劃。

02

預算分析與智慧調整

提供費用整合、預算控管、超支提醒及替代方案推薦，提升決策品質。

03

外部 API 智慧整合

即時整合地圖、天氣、餐廳及電影資訊，提升推薦精準度與情境判斷能力。

04

一站式服務整合

整合訂房、訂票與訂位服務，打造 LINE 群組內即可完成的智慧預訂平台。

01 景點排序

依據偏好、距離與熱門度，智慧排序景點

- 1  台北101
★★★★★
- 2  中正紀念堂
★★★★★
- 3  西門町
★★★★★
- 4  國立故宮博物院
★★★★★
- 5  象山步道
★★★★★

-  使用者偏好
文化、夜景、美食
-  景點距離
由近到遠排序
-  熱門程度
結合人氣與評分

03 停留時間估算

根據景點特性與使用者偏好，估算建議停留時間

- | | | |
|--|---------|----------|
|  | 台北101 | 🕒 90 分鐘 |
|  | 中正紀念堂 | 🕒 60 分鐘 |
|  | 西門町 | 🕒 120 分鐘 |
|  | 國立故宮博物院 | 🕒 120 分鐘 |
|  | 象山步道 | 🕒 60 分鐘 |

-  使用者偏好
輕鬆行程
-  景點特性
歷史、展覽、戶外
-  彈性調整
可依需求調整

02 路徑最佳化

運用演算法計算最佳路線，減少移動時間



-  總距離
18.7 km
-  預估總移動時間
45 分鐘
-  路線優化
減少 28% 移動時間

04 交通方式整合

整合多種交通工具，提供最佳搭配方案



- ☒ 即時交通資訊
- ☒ 票價預估
- ☒ 轉乘建議
- ☒ 時間預估

05 輸出完整行程表

輸出完整行程，清楚掌握時間、地點與交通

- | | | |
|-------|---|--|
| 08:30 |  | 出發地：台北車站
捷運板南線 → 台北101/世貿站
25 分鐘 |
| 09:00 | 1  | 台北101
停留 90 分鐘 |
| 10:30 |  | 步行
約 10 分鐘 |
| 10:40 | 2  | 中正紀念堂
停留 60 分鐘 |
| 12:00 |  | 公車 204
約 20 分鐘 |
| 12:20 | 3  | 西門町
停留 120 分鐘 |
| 14:20 |  | 捷運淡水信義線
約 25 分鐘 |
| 14:45 | 4  | 國立故宮博物院
停留 120 分鐘 |
| 16:45 |  | 公車 255
約 20 分鐘 |
| 17:15 | 5  | 象山步道
停留 60 分鐘 |
| 18:15 |  | 返回：台北車站 |

01

費用資料整合

整合各項費用資料，建立完整費用項目

	餐飲費	\$ 2,850
	交通費	\$ 1,150
	住宿費	\$ 3,600
	門票費	\$ 1,200
	娛樂費	\$ 900
	其他費用	\$ 300



預估總費用
\$ 10,000

02

預算限制比對

比對使用者設定的預算限制，判斷是否超支

設定預算
\$ 9,000 /人

預估總費用
\$ 10,000 /人

差額
+\$ 1,000



預算超支
超出預算 11.1%

03

花費結構分析

分析各項費用比例，掌握預算分配狀況



	餐飲費	\$ 3,600	36%
	住宿費	\$ 2,800	28%
	交通費	\$ 1,500	15%
	門票費	\$ 1,200	12%
	娛樂費	\$ 600	6%
	其他費用	\$ 300	3%

04

建議與調整機制

提供超支提醒與替代方案，協助調整預算

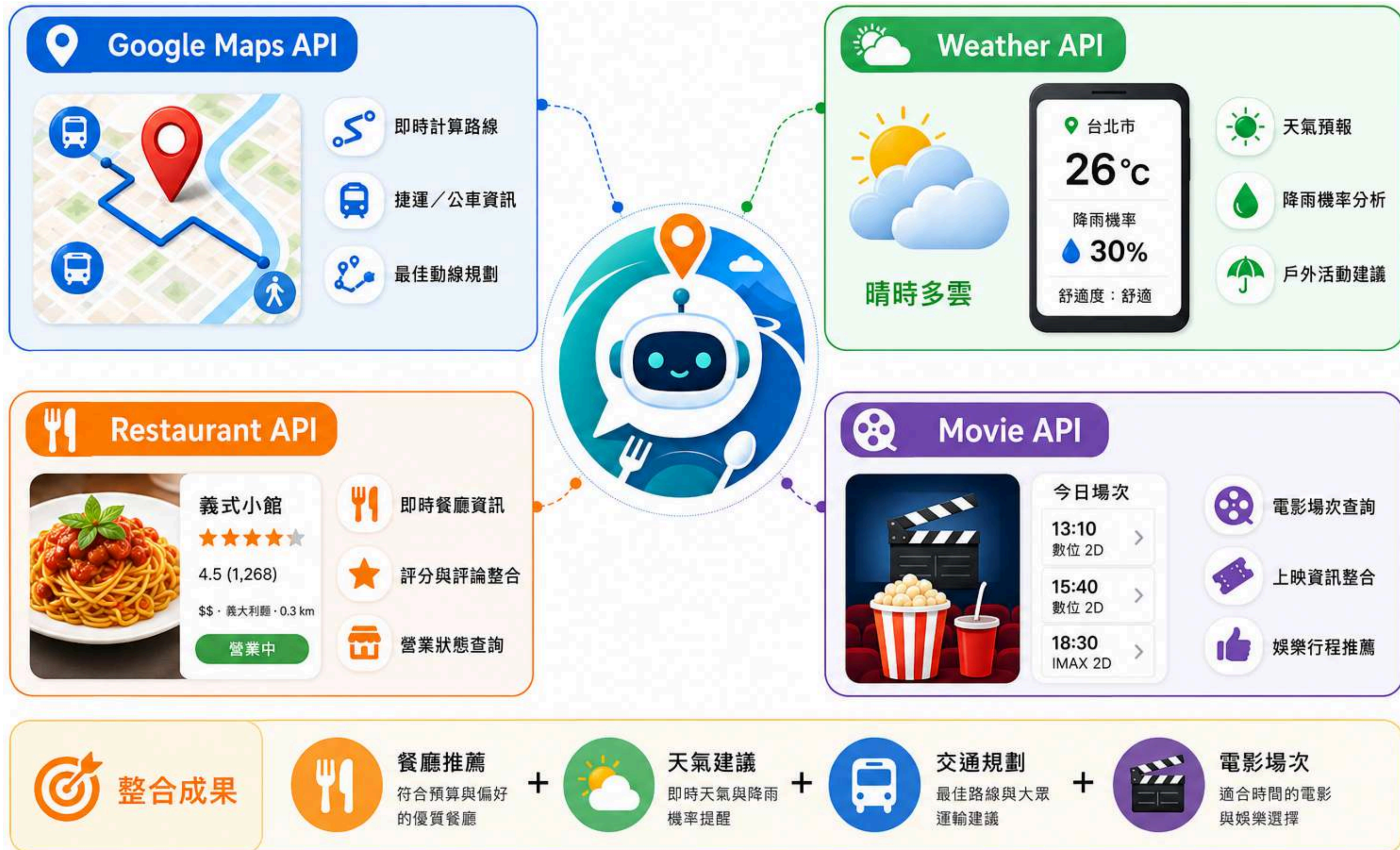
預算超支 \$1,000
以下提供替代方案，協助調整預算

	餐廳 替代方案	原方案 \$3,600	→	替代方案 \$2,800 ▼ \$800
	住宿 替代方案	原方案 \$2,800	→	替代方案 \$2,200 ▼ \$600
	交通 替代方案	原方案 \$1,500	→	替代方案 \$1,100 ▼ \$400

調整後預估
\$ 8,900 /人

符合預算

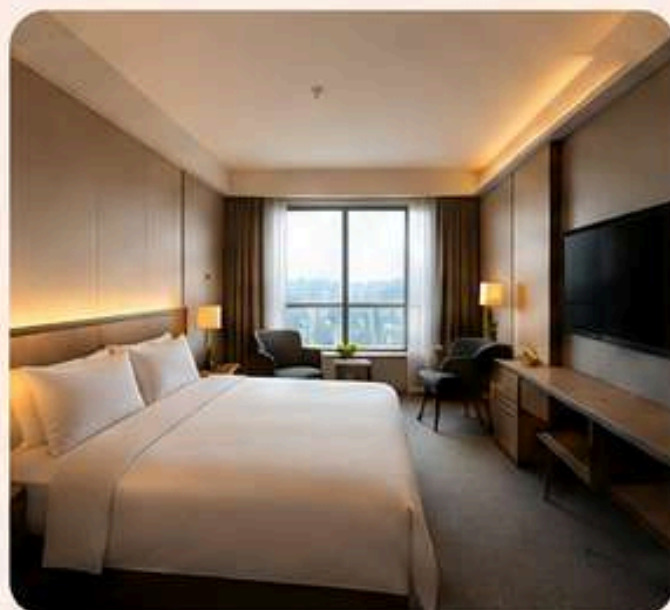

剩餘預算
\$ 100 /人





訂房

多種房型選擇，即時確認房況與價格



豪華雙人房

\$2,200 / 晚

2 人入住

1 大床

入住日期
2025/06/20

立即預訂



訂機票

多家航空比價，快速選擇最佳航班



台北 (TPE) → 東京 (NRT)

2025/06/20 (五)

09:30 直飛 13:30
TPE 約 3 小時 NRT

經濟艙 **\$6,880** / 人

立即訂票



餐廳訂位

精選餐廳推薦，線上訂位免排隊



義式小館

★★★★★ 4.6

用餐日期 2025/06/20

用餐時間 18:30

用餐人數 4 位

立即訂位



訂電影票

熱門電影場次查詢，線上選位快速購票



2025/06/20 (五)

19:00

影城：信義威秀影城

影廳：8 廳

票種：全票 × 2

座位：F8 F9

立即購票



安全便利

LINE 內完成交易
安全又放心



節省時間

一次完成預訂
無需跳轉網站

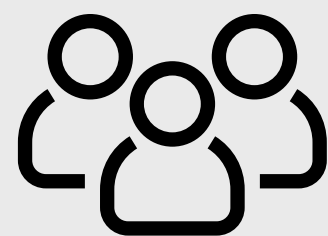
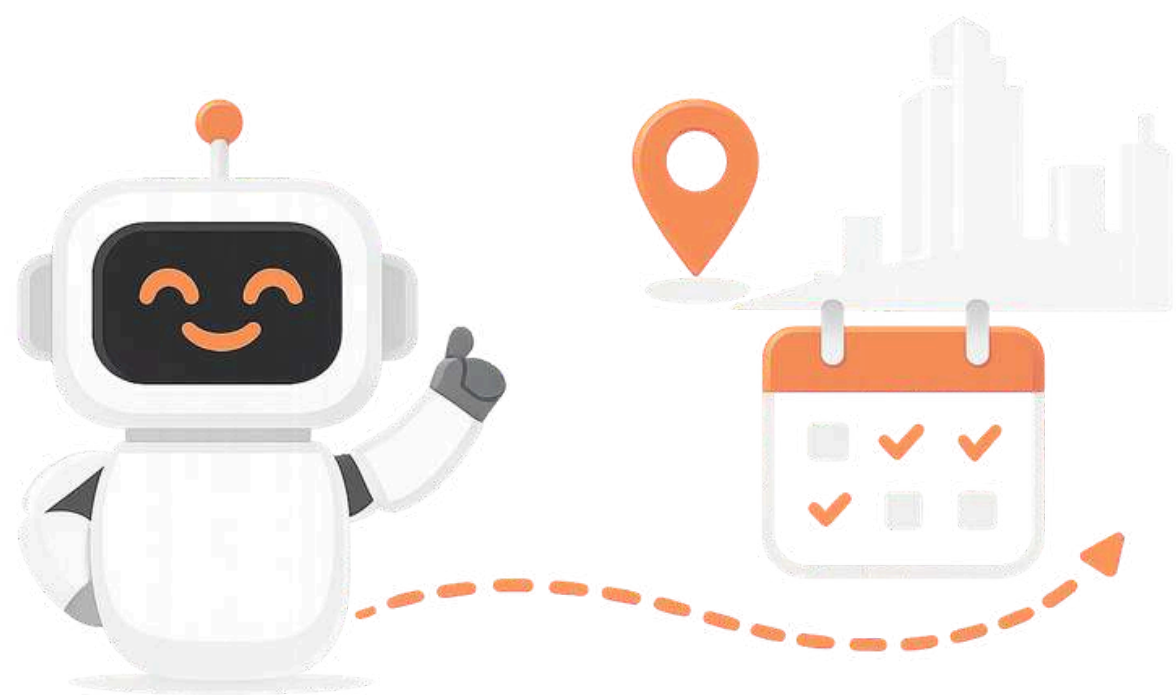


管理更簡單

所有訂單集中
查詢更方便

結論

本系統結合情境感知與 AI 技術，提出一套應用於群組決策情境之智慧推薦系統，並驗證本系統於實際群組情境中的應用可行性。



整合情境感知

透過自然語言理解，偵測使用者需求與情境，提供即時且合適的推薦。



提升決策效率

自動整合資訊、生成行程與預算建議，強化群組決策支援能力。



最佳化行程體驗

依據時間、交通與偏好，推薦最佳路線與景點，讓出遊與聚餐更順暢愉快。



改善資訊分散與決策冗長問題

結合 LINE Bot 的便利性與 AI 的智慧，打造貼近生活需求的智慧推薦助手。

2026 第27屆 永續發展管理研討會

The 27th Conference on Management in Sustainable Development

口頭發表證明

論文編號：SDMC-SS15-O-01

論文名稱：基於情境感知之 LINE Bot 智慧聚餐與
出遊推薦系統設計

論文作者：蒯思齊、蕭安庭、鄭喬尹、李若妘、
吳文鈺、賴若禾

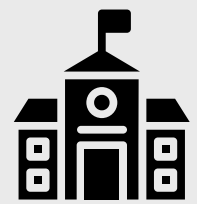
發表日期：115 年 05 月 22 日

國立屏東科技大學
管理學院 院長 陳 灯 能

中華民國 115 年 05 月 22 日



Thank you for listening



四技 第115410組



指導教授：蒯思齊 老師



組長：11246080 蕭安庭

組員：11246061 鄭喬尹

11246066 賴若禾

11246069 吳文鈺

11246079 李若妘



2026/6/12

